Physically Consistent Vision Generation

Lecture 20

Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

10 November, 2026

What this lecture covers

- Image-to-3D generation
- DiffusionGS
- Results of DiffusionGS
- Physics in text-to-video
- PhyGDPO
- Results of PhyGDPO
- Summary

Sources, figures and colors

The lecture presents two methods. DiffusionGS generates a 3D object or scene from one image (Cai et al., 2025a). PhyGDPO, physics-aware groupwise direct preference optimization, fine-tunes a text-to-video model so that its videos obey physics better (Cai et al., 2025b). The formulas and the numbers come from these two papers.

Pictures taken from a paper say so in their caption. The slides also play animations and videos; the notes show frames of them, and the captions say at what point of the clip each frame was taken.

In the block diagrams drawn for these notes, light grey boxes hold data (input views, rendered views, training videos), orange boxes are the steps done by a large network (the transformer of DiffusionGS, the vision-language models of the data pipeline), and white boxes are the other steps. In the tables, the best value of each column is in **red**. Other pictures use colors only to tell elements apart; their captions say what the colors mark.

Image-to-3D generation turns one view into a 3D model

Image-to-3D generation starts from a single photograph of an object or a scene, the *prompt view*, and produces a 3D representation of it: a description of the geometry and the appearance from which an image can be computed for any camera. An image computed for a camera that did not take the prompt view is a *novel view*.

The task is ill-posed. One image shows only the visible side of an object, so the back and the hidden parts have to be generated from what a model has learnt about objects in general. A generative model is therefore needed: it should produce a plausible 3D object that agrees with the prompt view, and different plausible objects for different random draws.

The representation used here is a cloud of 3D Gaussians: soft, colored blobs in space, each with a position, a size and shape, an opacity and a color, which are rendered to novel views. The applications include augmented and virtual reality (AR/VR), film making, robotics, animation and gaming, where 3D assets are expensive to model by hand.

Image-to-3D generation turns one view into a 3D model

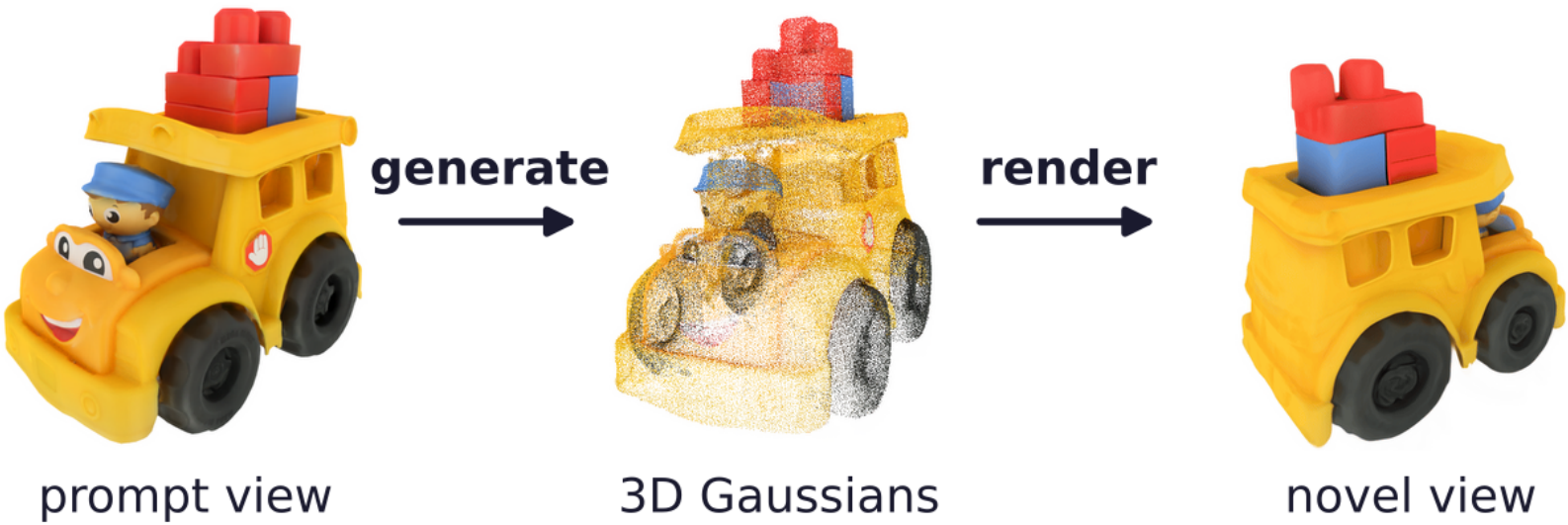


Figure 1: The task on one example: the prompt view of a toy truck, the 3D Gaussians generated from it (each dot is the center of one Gaussian, colored by the Gaussian's color), and a novel view rendered from them. On the slide the novel view is an animation of the camera circling the truck; this is one frame of it.

3D Gaussian splatting renders faster than NeRF

Two 3D representations are common in recent work. A neural radiance field (NeRF; Mildenhall et al., 2020) is a multilayer perceptron (MLP, a fully connected network) F_Θ that maps a 3D position and a viewing direction to a color and a density. A pixel is rendered by evaluating F_Θ at many points along its ray and summing the colors, each weighted by how much light reaches its point (volume rendering). With many network evaluations per pixel, rendering is slow and the resolution limited, and NeRF is hard to scale up to large scenes.

3D Gaussian splatting (3DGS; Kerbl et al., 2023) represents the scene explicitly by many 3D Gaussians. Each has a center $\boldsymbol{\mu}$, a covariance $\boldsymbol{\Sigma}$ (its size, shape and orientation), an opacity α and a color $\mathbf{c}$. To render, every Gaussian is projected onto the image as a 2D Gaussian (it is "splatted"), and the projected Gaussians are blended in depth order, in parallel on the graphics hardware: fast, and at high resolution. The Gaussians start at the points found by structure from motion (SfM, 3D points triangulated from many photographs) and are refined, split and pruned while they are fitted to the photographs.

3D Gaussian splatting renders faster than NeRF

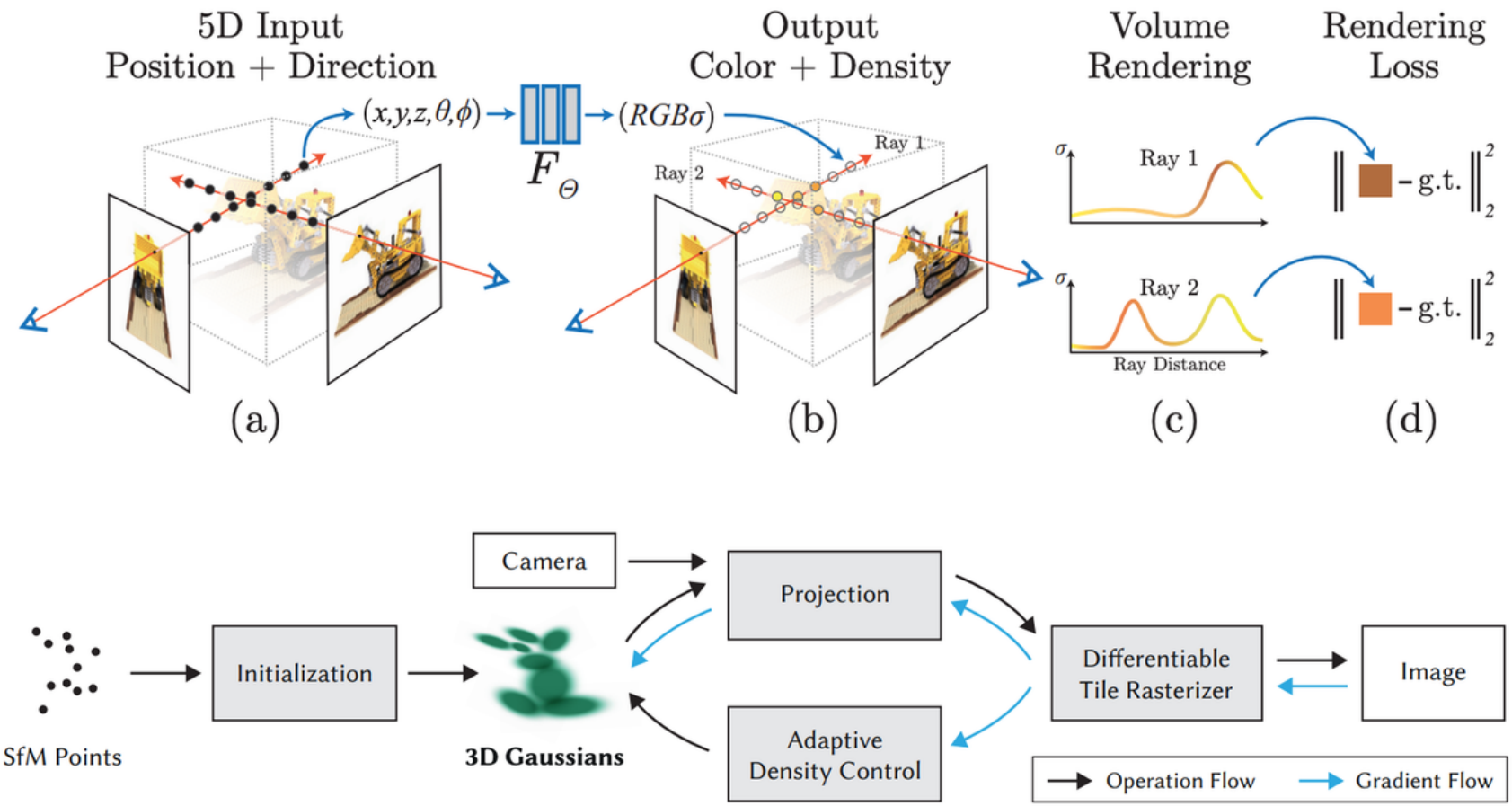


Figure 2: NeRF (Mildenhall et al., 2020, Figure 2), the upper diagram: (a) points on camera rays, with position and view direction, (b) F_Θ gives color and density, (c) volume rendering, (d) the loss against the true pixel (g.t.). 3DGS (Kerbl et al., 2023, Figure 2), the lower diagram: SfM points start the Gaussians, which are projected and rasterized; adaptive density control adds and removes Gaussians.

Existing methods generate views first, then reconstruct 3D

Most image-to-3D methods work in two stages. The first stage is a multi-view diffusion model, a network that generates images by removing noise from a random start step by step; it generates images of the object from several fixed viewpoints around it. The second stage reconstructs a 3D model from these images: CAT3D (Gao et al., 2024) fits a NeRF to them, and LGM, the large multi-view Gaussian model (Tang et al., 2024a), is a network that outputs 3D Gaussians from a few views.

The 3D model is then only reconstructed from generated 2D images, and the generator itself knows nothing about 3D. This raises the question whether such a method still generates 3D, and it points to the main limitation of the route: the 3D model can only be as consistent as the generated views.

Existing methods generate views first, then reconstruct 3D

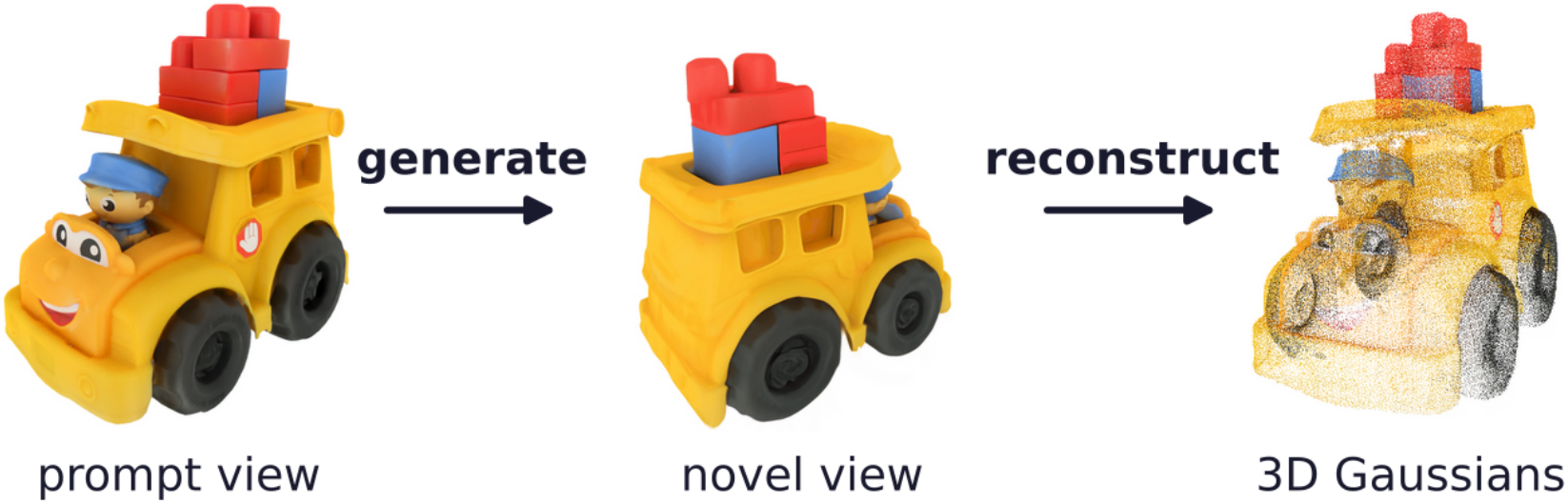


Figure 3: The two-stage route drawn on the toy truck: novel views are generated from the prompt view, and the 3D Gaussians are reconstructed from them. The images stand in for the views and the Gaussians of a two-stage method.

Objects come out view-inconsistent; scenes often collapse

The views of the first stage are generated together but are not tied to one 3D object, so they disagree in their details: a part is thicker in one view than in another, or appears in one view only. A reconstruction from views that disagree contains the disagreement as artifacts. Lifting such views to 3D easily leads to severe view inconsistency.

Scenes raise a second problem. Methods that reconstruct a scene from a single view rely on the depth of every pixel, from a separate depth estimator or predicted by the network itself. The depth places the visible surfaces, and everything the prompt view does not show must be invented. When the viewpoint changes significantly, these methods easily collapse: the novel view is blurred or torn. VistaDream (Wang et al., 2025) and Splatter Image (Szymanowicz et al., 2024) are examples.

DiffusionGS answers both problems with a one-stage diffusion model that generates 3D Gaussians directly, for objects and for scenes.

Objects come out view-inconsistent; scenes often collapse



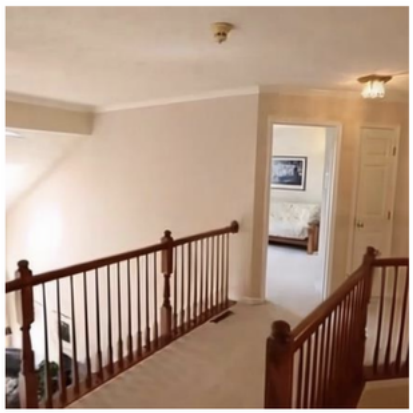
prompt view



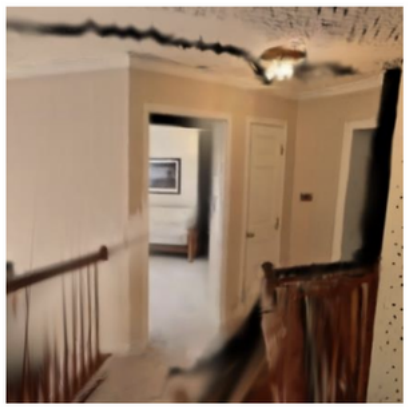
LGM



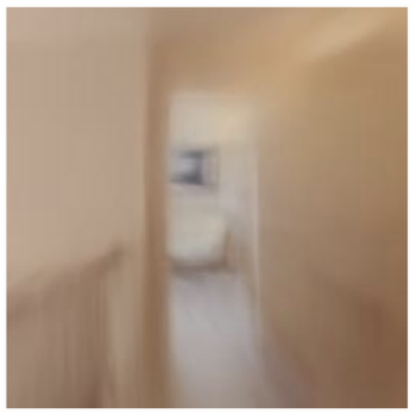
One-2-3-45++



prompt view



VistaDream



Splatter Image

Figure 4: A wall lamp, its prompt view and novel views by LGM and One-2-3-45++ (Liu et al., 2024): the shades and arms change shape. A hallway, its prompt view and novel views by VistaDream and Splatter Image: the railing is torn, the second view blurred.

Previous methods lack 3D ground truth and predict noise

The design of DiffusionGS starts from two questions about previous methods.

Why do previous methods generate multiple views instead of the 3D representation? Training a generator of 3D Gaussians directly would need many examples of the 3D representation itself, the 3D ground truth. Such data are very few and not accessible: 3D Gaussians are fitted to many photographs of each object, which few datasets provide. Photographs and renderings are plentiful.

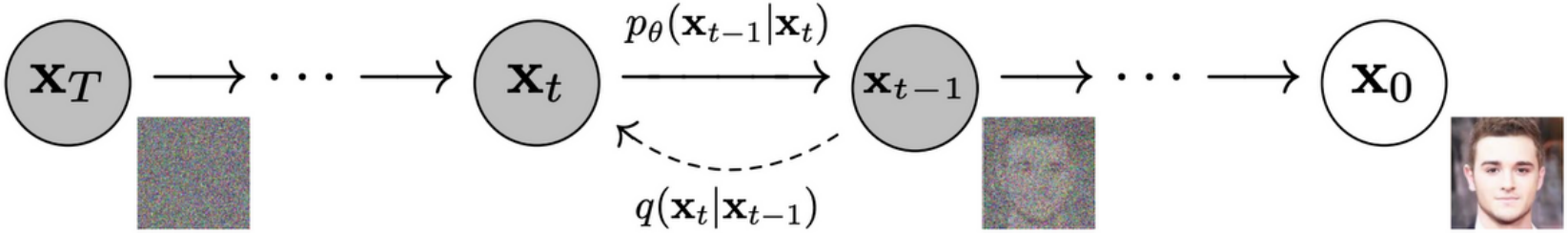
Why the noise? A diffusion model is usually trained to predict the noise ϵ that was added to an image; for an image the noise and the clean image carry the same information, since one follows from the other. In DiffusionGS the noise is added to the views, and the network outputs 3D Gaussians, whose rendering is an image of an object or a scene, never a noise pattern. The network therefore predicts the clean views, through its Gaussians.

DiffusionGS therefore predicts clean 3D Gaussians and supervises their renderings with images, so that it needs no 3D ground truth.

Previous methods lack 3D ground truth and predict noise



3D ground truth: very few, not accessible



the chain of the vanilla diffusion model (it predicts the noise)

Figure 5: The Gaussian point cloud of the toy truck, crossed out: 3D ground truth of this kind is not available for training. Under it, the chain of a diffusion model (Ho et al., 2020, Figure 2). The forward process q (dashed arrow) adds noise step by step from the clean image $\mathbf{x_0}$ (a face) to pure noise $\mathbf{x_T}$; the network p_θ (solid arrows) removes it step by step.

Review: the vanilla diffusion model adds noise step by step

A diffusion model (Ho et al., 2020) learns a distribution of images by learning to undo a gradual corruption. The forward process starts from a real image $\mathbf{x}_0 \sim q(\mathbf{x})$ and at each step $t = 1, \ldots, T$ scales it down a little and adds a little Gaussian noise, with a variance β_t set by a schedule. Write $\bar{\alpha}_t = \prod_{s \le t}(1 - \beta_s)$. The noisy image at step t then has a closed form:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\ (1 - \bar{\alpha}_t)\mathbf{I}\big),$$

that is, $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\epsilon}_t$ with $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. $\bar{\alpha}_t$ falls from 1 at $t = 0$ to nearly 0 at $t = T$. If the image has unit variance, $\bar{\alpha}_t$ is the share of the variance of $\mathbf{x}_t$ that comes from $\mathbf{x}_0$; at $t = T$ the image is nearly pure noise, $\mathbf{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$.

A 3D diffusion framework built on this needs two changes: supervision without 3D ground truth, and prediction of the clean data instead of the noise.

Review: the vanilla diffusion model adds noise step by step

The denoising process runs the chain backwards. A network with parameters θ defines $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ and is applied from pure noise $\mathbf{x}_T$ down to $\mathbf{x}_0$. In the vanilla model the network learns to predict the noise: from $\mathbf{x}_t$ and t it outputs $\hat{\epsilon}$, trained by

$$\mathbb{E}_{\mathbf{x_0}, t, \boldsymbol{\epsilon_t}} \left\| \boldsymbol{\epsilon}_t - \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t) \right\|^2.$$

The predicted noise gives a prediction of the clean image,

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\, \hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}},$$

so for images predicting ϵ and predicting $\mathbf{x}_0$ are equivalent (x_0-prediction versus ϵ-prediction). DiffusionGS uses x_0-prediction: its network outputs 3D Gaussians, and their rendered images play the part of $\hat{\mathbf{x}}_0$.

The denoiser predicts pixel-aligned 3D Gaussians

The network of DiffusionGS, the *denoiser*, takes the views at one timestep t and outputs 3D Gaussians. Its inputs are one clean condition view (the prompt view) and N noisy views. In training these are views of the object or scene from other cameras with noise of level t added; at test time they start as pure noise at N chosen camera poses. Each view comes with its camera, encoded for every pixel by the ray through that pixel (the reference-point Plücker coordinate, RPPC, a point of the ray and its direction). A linear layer maps each patch of a view to a token, a feature vector, with a positional embedding for its place. L transformer blocks, each a self-attention layer and an MLP, let every token attend to the tokens of all views. The timestep t enters every block through adaptive layer normalization: an MLP maps the embedding of t to the scale and shift of each normalization.

A decoder maps each patch token to one 3D Gaussian for each pixel of its patch; there are two decoders, for scenes and for objects, which have different ranges of depth. The Gaussians are rasterized to the views, giving the denoised views, and the losses compare these with the clean views.

The denoiser predicts pixel-aligned 3D Gaussians

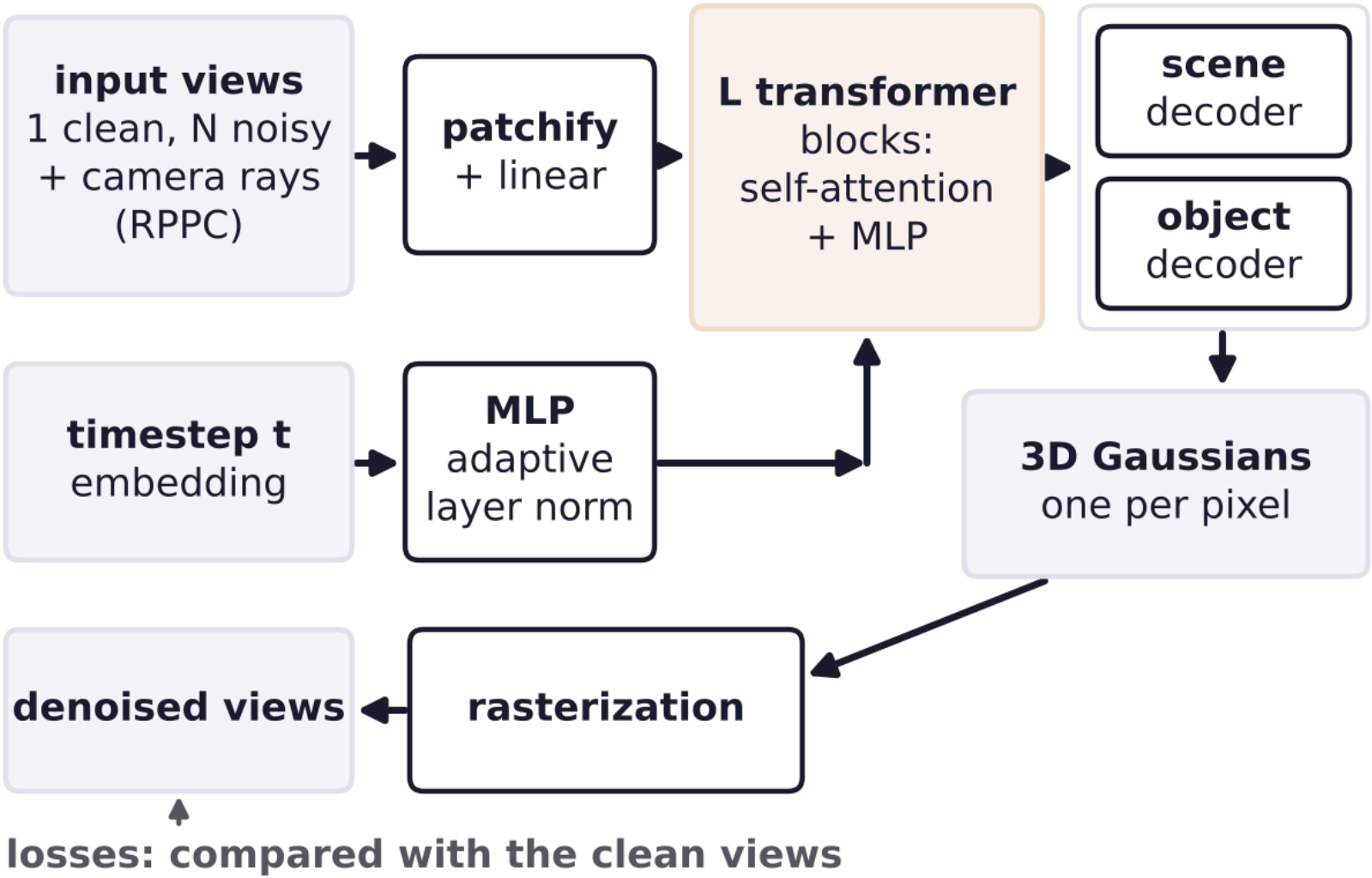


Figure 6: The denoiser at one timestep, drawn after Cai et al. (2025a), Figure 4(b), from the inputs to the denoised views; L is the number of transformer blocks. Grey: data; orange: the transformer; white: the other steps.

At every timestep the denoiser outputs 3D Gaussians

At timestep t the inputs are the condition view $\mathbf{x}_{con}$ with its viewpoint $\mathbf{v}_{con}$, and the N noisy views with their viewpoints,

$$\mathcal{X}_t = \{\mathbf{x}_t^{(1)}, \ldots, \mathbf{x}_t^{(N)}\}, \qquad \mathcal{V} = \{\mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(N)}\}.$$

The noisy views are the clean views $\mathcal{X}_0 = \{\mathbf{x}_0^{(1)}, \ldots, \mathbf{x}_0^{(N)}\}$ after the forward process,

$$\mathbf{x}_t^{(i)} = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0^{(i)} + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_t^{(i)}.$$

The denoiser outputs a set of 3D Gaussians,

$$\mathcal{G}_\theta(\mathcal{X}_t \,|\, \mathbf{x}_{con}, \mathbf{v}_{con}, t, \mathcal{V}) = \left\{ G_t^{(k)}\left(\boldsymbol{\mu}_t^{(k)}, \boldsymbol{\Sigma}_t^{(k)}, \alpha_t^{(k)}, \mathbf{c}_t^{(k)}\right)\right\}, \qquad 1 \le k \le N_g,$$

each with a center, a covariance, an opacity and a color (the opacity α is unrelated to the schedule $\bar{\alpha}_t$). There is one Gaussian for each pixel of each view, condition view included, so that with images of $H \times W$ pixels $N_g = (N+1)HW$.

At every timestep the denoiser outputs 3D Gaussians

The Gaussians are *pixel-aligned*: the Gaussian of a pixel sits on the ray of that pixel. With $\mathbf{o}^{(k)}$ the center of the camera and $\mathbf{d}^{(k)}$ the direction of the ray,

$$\boldsymbol{\mu}_t^{(k)} = \mathbf{o}^{(k)} + u_t^{(k)}\,\mathbf{d}^{(k)}, \qquad u_t^{(k)} = w_t^{(k)}\,u_{near} + \left(1 - w_t^{(k)}\right) u_{far},$$

so the network predicts a weight $w_t^{(k)}$ between 0 and 1 that places the Gaussian between a near and a far distance along its ray. Each view thus gives a per-pixel map of Gaussians, $\hat{\mathbf{H}}^{(i)}$, an image whose pixels hold Gaussians. Merging the primitives of all the maps,

$$\hat{\mathcal{H}} = \{\hat{\mathbf{H}}_{con}, \hat{\mathbf{H}}^{(1)}, \ldots, \hat{\mathbf{H}}^{(N)}\},$$

gives the point cloud.

Why per-pixel? The Gaussians are aligned with the pixels and regular: every pixel of every view contributes one, the network only has to predict a depth and the other parameters for it, and the point cloud covers what the views see.

Rendering the Gaussians gives the denoised views

The denoised image of view i, the estimate of the clean view $\mathbf{x}_0^{(i)}$ made at step t, is the rendering of the Gaussians with the camera of that view:

$$\hat{\mathbf{x}}_{(0,t)}^{(i)} = F_r\left(\mathbf{M}_{ext}^{(i)}, \mathbf{M}_{int}^{(i)}, \mathcal{G}_\theta(\mathcal{X}_t \mid \mathbf{x}_{con}, \mathbf{v}_{con}, t, \mathcal{V})\right).$$

The extrinsic matrix $\mathbf{M}_{ext}$ gives the position and orientation of the camera, the intrinsic matrix $\mathbf{M}_{int}$ its focal length and image center. The rasterizer F_r projects each 3D Gaussian into the image, where its covariance becomes $\mathbf{\Sigma}' = \mathbf{JW\Sigma W}^\mathsf{T}\mathbf{J}^\mathsf{T}$, with $\mathbf{W}$ the rotation into the camera frame (bold, unlike the image width W) and $\mathbf{J}$ the Jacobian of the projection.

Rendering the Gaussians gives the denoised views

The color of a pixel (m, n) blends the Gaussians that cover it, sorted from near to far:

$$\hat{\mathbf{x}}^{(i)}_{(0,t)}(m,n) = \sum_{j \in \mathcal{N}} \mathbf{c}_t^{(j)}\, \sigma_t^{(j)} \prod_{l=1}^{j-1} \left(1 - \sigma_t^{(l)}\right),$$

where $\mathcal{N}$, a second use of the letter, is the set of Gaussians that the pixel's ray meets, and $\sigma_t^{(j)}$ is the opacity $\alpha_t^{(j)}$ times the value of Gaussian j where the ray meets it. The product is the share of light that passes the Gaussians in front.

The point cloud is supervised at its renderings $\hat{\mathcal{X}}_{(0,t)}$:

$$\mathcal{L}_{de} = \mathcal{L}_2\left(\hat{\mathcal{X}}_{(0,t)}, \mathcal{X}_0\right) + \lambda \cdot \mathcal{L}_{\mathrm{VGG}}\left(\hat{\mathcal{X}}_{(0,t)}, \mathcal{X}_0\right),$$

where $\mathcal{L}_2$ is the squared difference of the pixels and $\mathcal{L}_{\mathrm{VGG}}$ a perceptual loss: the difference of the features that a VGG-19 network, trained to classify images, computes from the two images; λ weights the two terms. The clean views are photographs or renderings, so no 3D ground truth is needed.

Rendering the Gaussians gives the denoised views

At test time the N noisy views start as pure noise at N pre-designed camera poses, and DiffusionGS runs 30 steps of DDIM, the denoising diffusion implicit model sampler (Song et al., 2021), with the update

$$\mathbf{x}_{t-1}^{(i)} = \sqrt{\bar{\alpha}_{t-1}}\,\hat{\mathbf{x}}_{(0,t)}^{(i)} + \sqrt{1-\bar{\alpha}_{t-1}}\,\boldsymbol{\epsilon}_{t-1}^{(i)},$$

where $t-1$ stands for the next of the 30 timesteps. The rendering estimates the clean view, and the update returns to the next noise level. In deterministic DDIM, $\boldsymbol{\epsilon}_{t-1}^{(i)}$ is the noise implied by $\mathbf{x}_t^{(i)}$ and the estimate, $(\mathbf{x}_t^{(i)} - \sqrt{\bar{\alpha}_t}\,\hat{\mathbf{x}}_{(0,t)}^{(i)})/\sqrt{1-\bar{\alpha}_t}$. Only one clean image and the relative camera poses are required.

Scene-object mixed training constrains the viewpoints

3D training data are scarce and expensive to annotate. Scene datasets have only about 90K training samples, which cover small viewpoint changes, too few to learn strong geometry. DiffusionGS therefore trains on objects and scenes together, from Objaverse (Deitke et al., 2023) and MVImgNet (Yu et al., 2023a) for objects and RealEstate10K (Zhou et al., 2018) and DL3DV-10K (Ling et al., 2024) for scenes.

The two kinds of data differ. Object data show one object centered on a blank background, within a small range of depth; scene data have dense backgrounds and a much wider range of depth. Training on both directly may introduce artifacts or fail to converge. Two measures make them compatible. First, the cameras of each training example are constrained: seen from the center of the scene, the angles between their positions and between their viewing directions must stay below thresholds, so that the views of one example overlap and share enough content for the training to converge. Second, a dual Gaussian decoder serves the different imaging ranges of objects and scenes: each of its two MLPs places the Gaussians within the depth range of its own kind of data, $[0.1, 4.2]$ for objects and $[0, 500]$ for scenes.

Scene-object mixed training constrains the viewpoints

The constraints are

$$\theta_{cd}^{(i)} \leq \theta_1, \quad \theta_{dn}^{(i,j)} \leq \theta_2, \quad \frac{\vec{z}_{con} \cdot \vec{z}_{noise}^{(i)}}{|\vec{z}_{con}|\,|\vec{z}_{noise}^{(i)}|} \geq \cos\varphi_1, \quad \frac{\vec{z}_{con} \cdot \vec{z}_{nv}^{(j)}}{|\vec{z}_{con}|\,|\vec{z}_{nv}^{(j)}|} \geq \cos\varphi_2.$$

The angles θ are measured at the world origin O_w, the center of the object or scene. $\theta_{cd}^{(i)}$ is the angle between the positions of the condition camera and of the i-th noisy-view camera; $\theta_{dn}^{(i,j)}$ the angle between the positions of the i-th noisy camera and of the j-th novel-view camera, a view used only to supervise the rendering. The vectors $\vec{z}$ are the forward directions of the cameras (con: condition, noise: noisy view, nv: novel view), and the cosine constraints keep every camera looking within an angle φ_1 or φ_2 of the condition camera's direction. θ_1, θ_2, φ_1 and φ_2 are thresholds set by hand. These angles θ are unrelated to the network parameters θ of $\mathcal{G}_\theta$.

Scene-object mixed training constrains the viewpoints

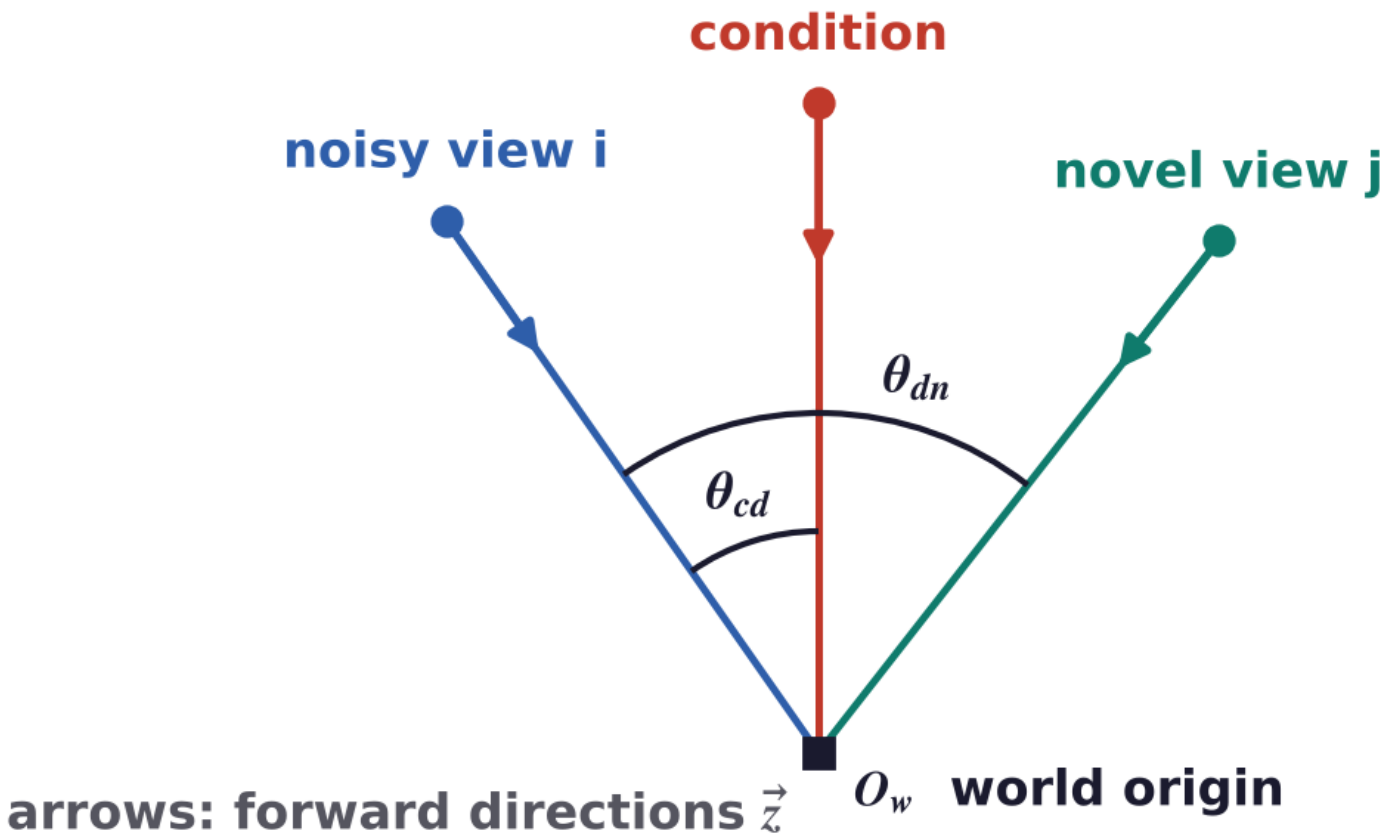


Figure 7: The position constraints in the plane, drawn after Cai et al. (2025a), Figure 4(a): the world origin (square), the condition camera (red), a noisy-view camera (blue) and a novel-view camera (teal). Every camera here faces O_w (arrows), so the angles φ between the forward directions equal the angles θ.

RPPC encodes the ray position and the relative depth

The camera of each view enters the network as one ray per pixel, from the camera center $\mathbf{o}$ along the unit direction $\mathbf{d}$: its points are $\mathbf{o} + s\,\mathbf{d}$, $s \geq 0$.

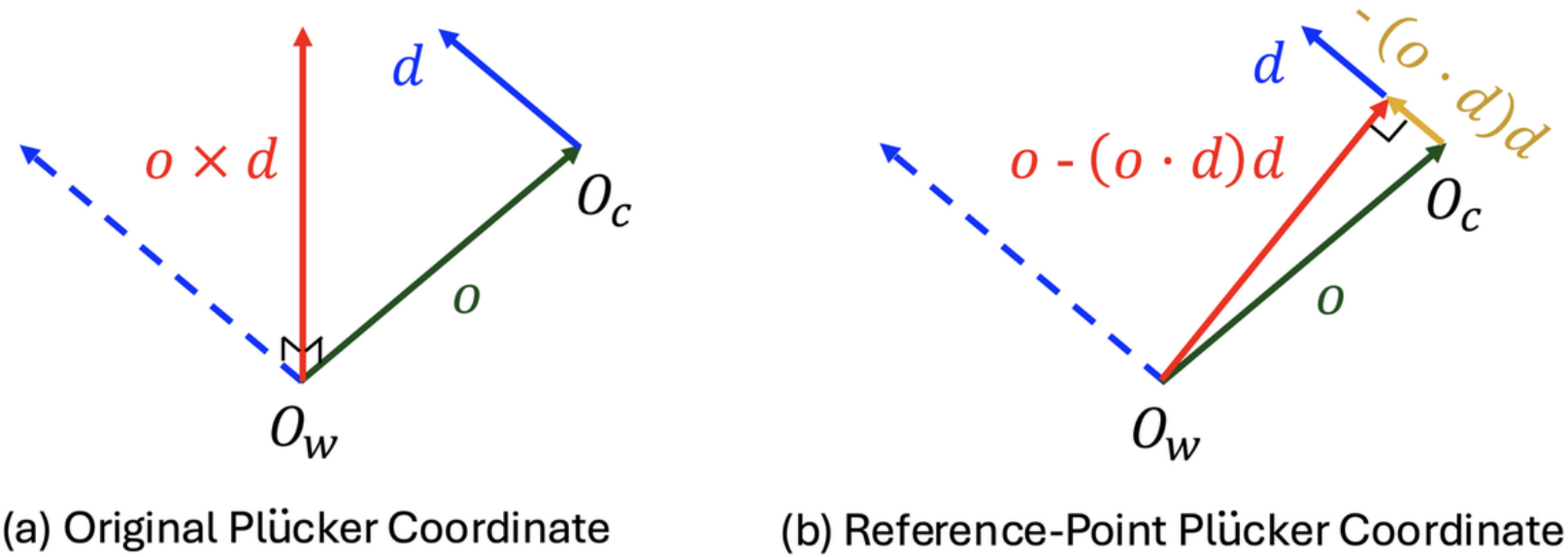
The usual encoding is the Plücker coordinate of the line, $\mathbf{r} = (\mathbf{o} \times \mathbf{d}, \mathbf{d})$ (Sitzmann et al., 2021). The moment $\mathbf{o} \times \mathbf{d}$ is the same for every point of the line: its length is the distance of the line from the origin, and it is normal to the plane through the origin and the line. It mainly represents the rotational effect of $\mathbf{o}$ about $\mathbf{d}$: where the ray passes stays implicit.

The reference-point Plücker coordinate (RPPC) replaces the moment by a point of the ray:

$$\mathbf{r} = \big(\mathbf{o} - (\mathbf{o} \cdot \mathbf{d})\,\mathbf{d},\ \mathbf{d}\big).$$

$\mathbf{o} - (\mathbf{o} \cdot \mathbf{d})\,\mathbf{d}$ is the point of the line closest to the world origin (its dot product with $\mathbf{d}$ is zero). Both encodings fix the same line, since this point equals $\mathbf{d} \times (\mathbf{o} \times \mathbf{d})$; RPPC hands it to the network directly, so the network reads off where the ray passes the center of the scene, and so the relative depth of the scene along the ray, without computing a cross product. By skip connections this information reaches every transformer block and the decoder.

RPPC encodes the ray position and the relative depth



(a) Original Plücker Coordinate

(b) Reference-Point Plücker Coordinate

Figure 8: Cai et al. (2025a), Figure 5. O_w: the world origin; O_c: the camera center; green: **o**; blue: **d**, and dashed, **d** drawn again at O_w. (a) Plücker: red, **o** × **d**, drawn in perspective, normal to the plane of **o** and **d**. (b) RPPC: red, **o** − (**o** · **d**)**d**, reached from O_c by the gold step −(**o** · **d**)**d**.

A point distribution loss warms up object generation

Object views are centered on the object, so the generated point cloud should be concentrated around the world origin. A point distribution loss encourages this at the start of training on object data, as a warm-up. With $l_t^{(k)} = |u_t^{(k)}\mathbf{d}^{(k)}|$ the distance of Gaussian k from its camera center,

$$\mathcal{L}_{pd} = \mathbb{E}_k\Big[\, l_t^{(k)} - \Big(\frac{l_t^{(k)} - \mathbb{E}_k[l_t^{(k)}]}{\sqrt{\mathrm{Var}(l_t^{(k)})}}\,\sigma_0 + \mathbb{E}_k\big[|\mathbf{o}^{(k)}|\big]\Big)\Big].$$

The term in brackets is a target for each distance: the distance standardized over all Gaussians, rescaled to the target standard deviation $\sigma_0 = 0.5$ (unrelated to the blending weights σ_t) and shifted to the mean distance $\mathbb{E}_k|\mathbf{o}^{(k)}|$ of the cameras from the origin; distances near it put the Gaussians around the origin. The overall training objective is

$$\mathcal{L} = (\mathcal{L}_{de} + \mathcal{L}_{nv}) \cdot \mathbf{1}_{iter > iter_0} + \mathcal{L}_{pd} \cdot \mathbf{1}_{iter < iter_0} \cdot \mathbf{1}_{object},$$

where $\mathcal{L}_{nv}$ is the rendering loss of $\mathcal{L}_{de}$ applied to the novel views, $iter_0$ is the end of the warm-up, and $\mathbf{1}_{object}$ restricts $\mathcal{L}_{pd}$ to object data.

RPPC also improves feed-forward CT reconstruction

The RPPC is a general way of telling a network about cameras. It is also used for a different problem, computed tomography (CT): the reconstruction of the 3D density of a body from X-ray images taken around it.

X-LRM, the X-ray large reconstruction model (Zhang et al., 2025), reconstructs a CT volume from a few X-ray views in one forward pass of a network, without the per-patient optimization of classical reconstruction (feed-forward reconstruction). Its encoder, the X-former, is a transformer that takes the X-ray images together with their RPPC ray embeddings; X-LRM adopts RPPC because it encodes more of the ray position and the relative depth than standard Plücker coordinates. A triplane decoder, also a transformer, turns the features into the X-triplane: three 2D feature planes from which a small network reads off the density at any 3D point. The predicted volume is compared with the true one.

RPPC also improves feed-forward CT reconstruction

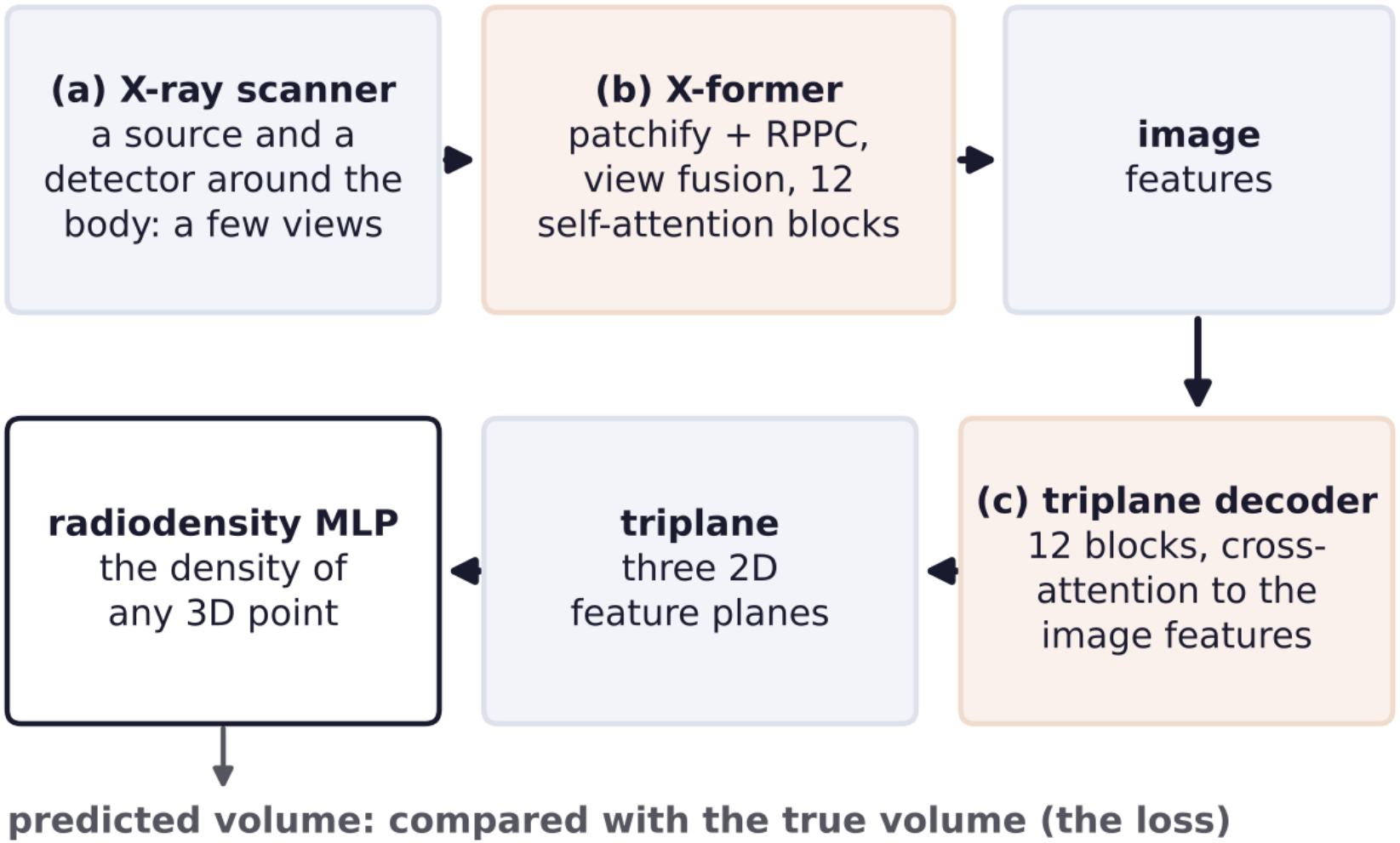


Figure 9: X-LRM, drawn after Zhang et al. (2025), Figure 2. Grey: data; orange: the two transformers; white: the MLP that reads the density (radiodensity) of each point.

DiffusionGS generates objects and scenes from a single view

From a single view DiffusionGS generates an object or a scene in about 6 seconds on one A100 GPU (graphics processing unit).



Figure 10: Four generated objects and, in the second row, two generated scenes: one frame of each animation of a moving camera.

More objects and scenes generated from a single view

More results, each rendered from a viewpoint other than the prompt view; the project page shows them as animations.



Figure 11: Four more generated objects and, in the second row, two more generated scenes: one frame of each animation.

DiffusionGS gives clearer novel views of objects

The comparison shows novel views of four test objects generated by DiffusionGS and five earlier image-to-3D methods: DMV3D (Xu et al., 2024), LGM, CRM (Wang et al., 2024), One-2-3-45++ and DreamGaussian (Tang et al., 2024b). The prompt views come from two datasets of 3D objects, ABO, Amazon Berkeley Objects (Collins et al., 2022), and GSO, Google Scanned Objects (Downs et al., 2022); from a real photograph; and from a text-to-image model.

The prompt view of the turtle shows its side and the novel views show it from above, so the methods have to generate the shell that the prompt view hides. DiffusionGS keeps fine details and a plausible 3D shape. The others lose detail: LGM blurs the lamp and the turtle, DMV3D smears the action figure, and DreamGaussian turns the lamp's cages into blobs and the shell into noise.

DiffusionGS gives clearer novel views of objects

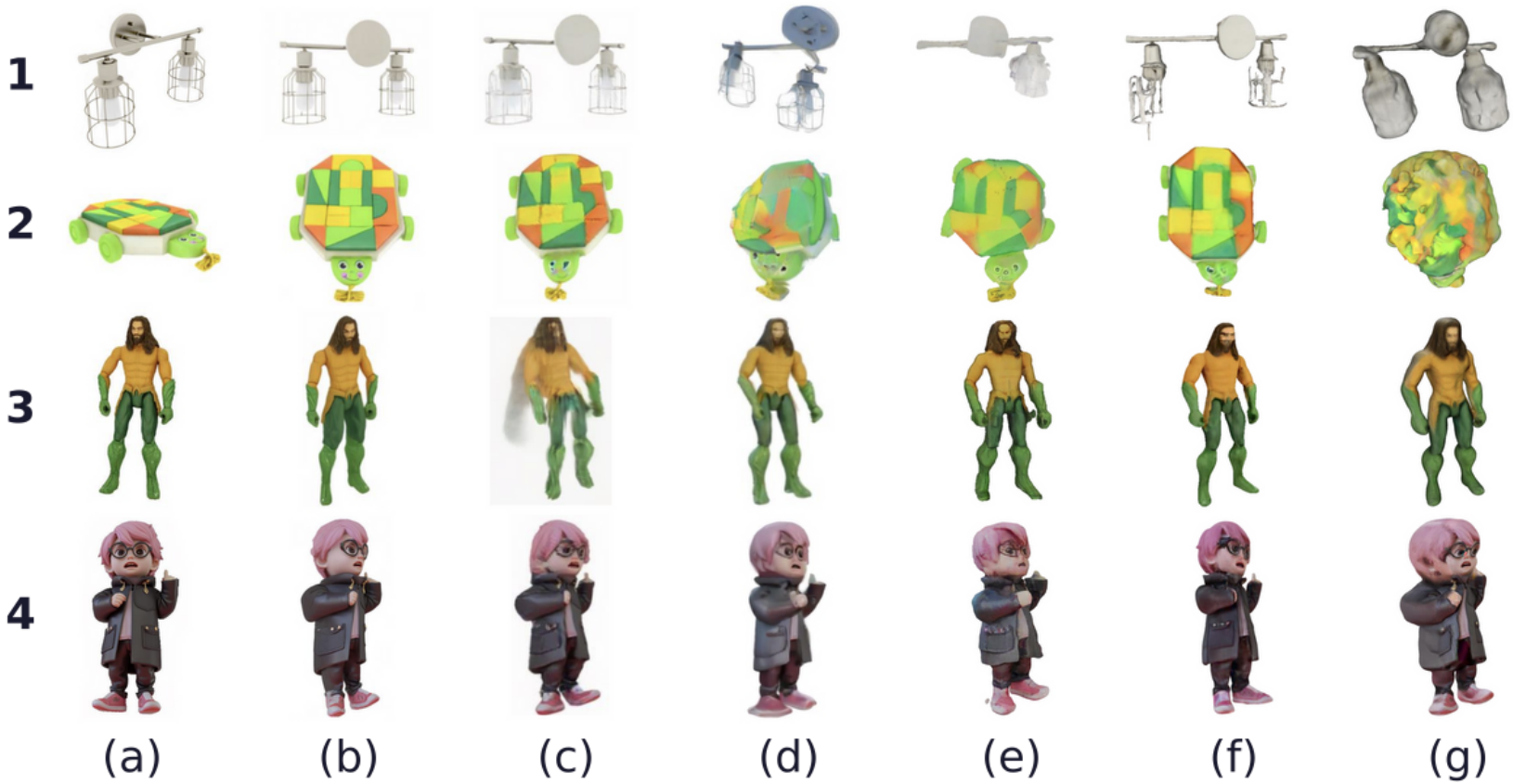


Figure 12: Novel views of four objects (Cai et al., 2025a, Figure 6). Rows: 1, a wall lamp from ABO; 2, a toy turtle from GSO; 3, an action figure in a real photograph; 4, a character generated from text. Columns: (a) the prompt view; (b) DiffusionGS; (c) DMV3D; (d) LGM; (e) CRM; (f) One-2-3-45++; (g) DreamGaussian.

DiffusionGS leads on ABO, GSO and in a user study

The novel views are compared with the true views of the test objects by four measures. PSNR, the peak signal-to-noise ratio, is $10 \log_{10}(1/\mathrm{MSE})$ for pixel values in $[0, 1]$, with MSE the mean squared error, in decibels (dB); higher is better. SSIM, the structural similarity, compares local means, variances and correlations of the two images; 1 is a perfect match. LPIPS, the learned perceptual image patch similarity, is a distance between the features of a deep network, close to human judgments; lower is better. FID, the Fréchet inception distance, compares the distribution of the generated images with that of real images, through the features of an Inception network; lower is better. GS-LRM, the Gaussian splatting large reconstruction model (Zhang et al., 2024), is a further baseline.

DiffusionGS is best on all four measures on both datasets, by 2.20 dB PSNR and 23.25 FID on ABO over the next best method, DMV3D. In a user study, where 25 people scored 14 objects from 1 (worst) to 6 (best), it scores highest. It takes 5.8 seconds per object, more than five times faster than DMV3D (31.4 s); only LGM is faster (4.1 s).

DiffusionGS leads on ABO, GSO and in a user study

	ABO				GSO			
method	PSNR↑	SSIM↑	LPIPS↓	FID↓	PSNR↑	SSIM↑	LPIPS↓	FID↓
LGM	16.01	0.7262	0.3255	86.32	14.27	0.7183	0.3003	75.55
GS-LRM	18.78	0.7974	0.2720	123.55	17.70	0.7950	0.2411	112.96
DMV3D	23.69	0.8634	0.1131	32.28	20.82	0.8347	0.1289	33.48
DiffusionGS	**25.89**	**0.8880**	**0.0965**	**9.03**	**22.07**	**0.8545**	**0.1115**	**11.52**

method	user study score (1–6)↑	running time (s)↓
DreamGaussian	1.94	120
LGM	3.04	**4.1**
DMV3D	3.16	31.4
CRM	2.69	10
One-2-3-45++	3.81	60.0
DiffusionGS	**4.88**	5.8

Object generation on ABO and GSO, and the user study with the running times (Cai et al., 2025a, Table 1 (a), (c), (d)).

DiffusionGS gives the best scene views on RealEstate10K

For scenes the test set is RealEstate10K, video clips of houses, inside and outside, with the camera of every frame. The prompt view is one frame and the true novel view another frame of the same clip, its ground truth. The baselines are single-view scene reconstruction methods: Flash3D (Szymanowicz et al., 2025), Splatter Image and PixelNeRF (Yu et al., 2021) in the table, and VistaDream in the pictures only. DiffusionGS is best on all four measures, by 1.34 dB PSNR and 19.16 FID over the next best, Flash3D.

RealEstate10K	PSNR↑	SSIM↑	LPIPS↓	FID↓
PixelNeRF	17.46	0.5713	0.5525	159.52
Splatter Image	18.21	0.6115	0.4839	120.35
Flash3D	20.29	0.6483	0.3610	35.03
DiffusionGS	**21.63**	**0.6787**	**0.2743**	**15.87**

Scene reconstruction on RealEstate10K (Cai et al., 2025a, Table 1 (e)).

DiffusionGS gives the best scene views on RealEstate10K



Figure 13: Novel views of two scenes (Cai et al., 2025a, Figure 7): a hallway at the top of a staircase and the back of a house. Columns: (a) the prompt view; (b) the ground truth; (c) DiffusionGS; (d) Flash3D; (e) VistaDream; (f) Splatter Image; (g) PixelNeRF. The railing and the corners stay straight in (c); (e) tears the railing, (f) and (g) are blurred.

DiffusionGS scores above PhotoNVS and needs no 3DGS fitting

A different route uses a 2D diffusion model that synthesizes novel views directly, without any 3D representation; PhotoNVS (Yu et al., 2023b) is such a method. A 3D model must then be fitted to the synthesized views afterwards (post-hoc 3DGS fitting), which takes long and inherits any inconsistency of the views. On RealEstate10K, DiffusionGS takes 6 s against 61 s for PhotoNVS plus 2417 s for the fitting, and it scores higher. Depth maps show the geometry: for PhotoNVS they come from the Gaussians fitted afterwards. Smooth depth that follows the walls means consistent geometry; DiffusionGS without RPPC gives a less accurate depth and structure.

method	inference	post-hoc 3DGS	PSNR↑	SSIM↑	LPIPS↓	FID↓
PhotoNVS	61 s	2417 s	15.31	0.5215	0.4589	28.30
DiffusionGS	**6 s**	**0 s**	**21.63**	**0.6787**	**0.2743**	**15.87**

Comparison with PhotoNVS on RealEstate10K (Cai et al., 2025a, Table 1 (b)).

DiffusionGS scores above PhotoNVS and needs no 3DGS fitting

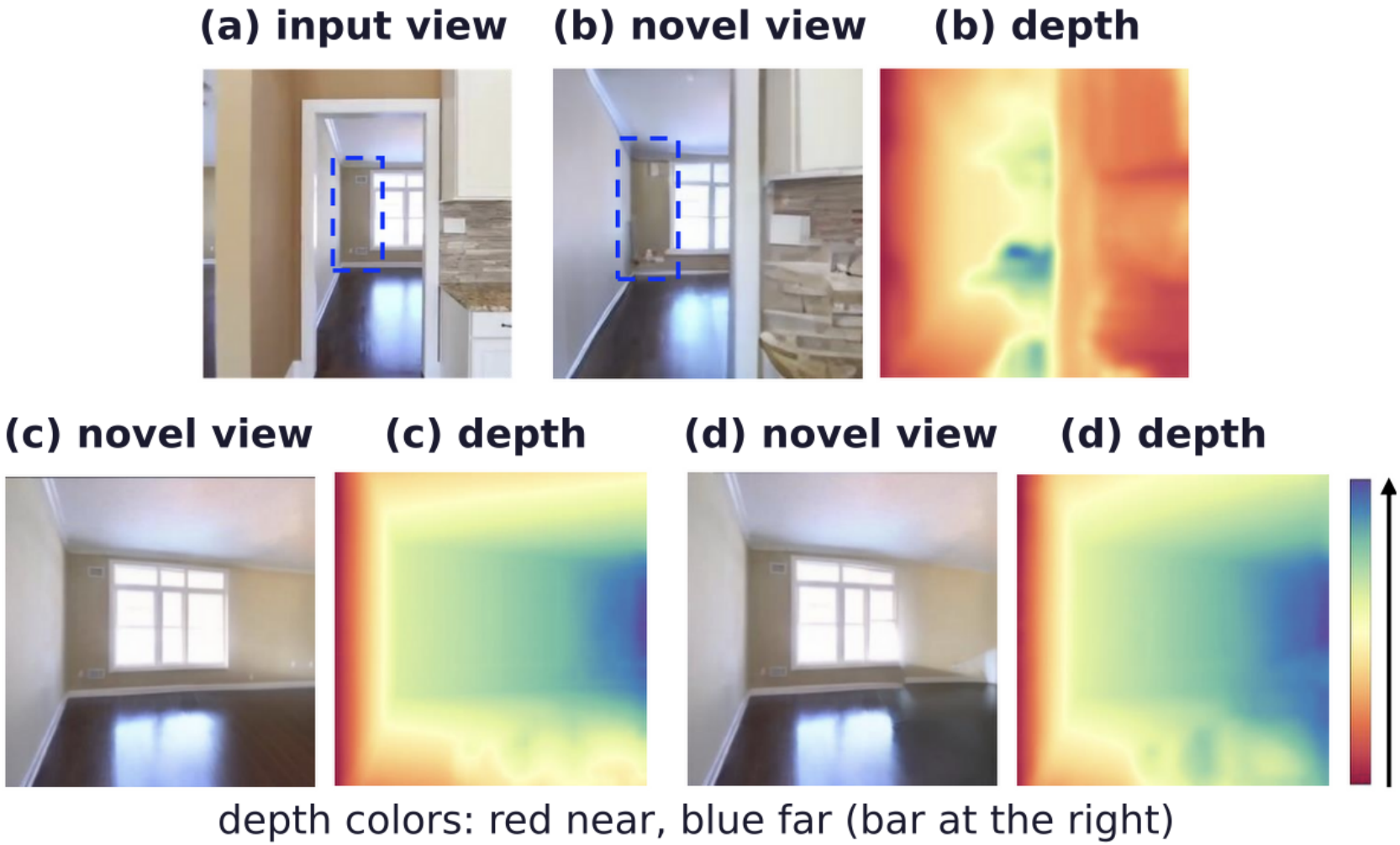


Figure 14: Cai et al. (2025a), Figure 8. (a) The input view; (b) PhotoNVS; (c) DiffusionGS; (d) DiffusionGS without RPPC. Blue boxes: content that PhotoNVS renders inconsistently.

Every component of DiffusionGS improves the results

The ablation study on GSO starts from a baseline, the denoiser without timestep control trained on objects, and adds the components of DiffusionGS one at a time: the diffusion framework that predicts 3D Gaussians, the point distribution loss $\mathcal{L}_{pd}$, the scene-object mixed training, and RPPC. Every measure improves at every step. FID falls by 70.45 with the diffusion framework, then by 19.45, 10.62 and 6.27; PSNR rises by 2.94 dB, then by 0.37, 0.79 and 0.34 dB.

	PSNR↑	SSIM↑	LPIPS↓	FID↓
baseline	17.63	0.7928	0.2452	118.31
+ Gaussian diffusion	20.57	0.8120	0.1417	47.86
+ point distribution loss	20.94	0.8423	0.1218	28.41
+ mixed training	21.73	0.8515	0.1196	17.79
+ RPPC	**22.07**	**0.8545**	**0.1115**	**11.52**

Ablation on GSO (Cai et al., 2025a, Table 2); each row adds one component.

Every component of DiffusionGS improves the results

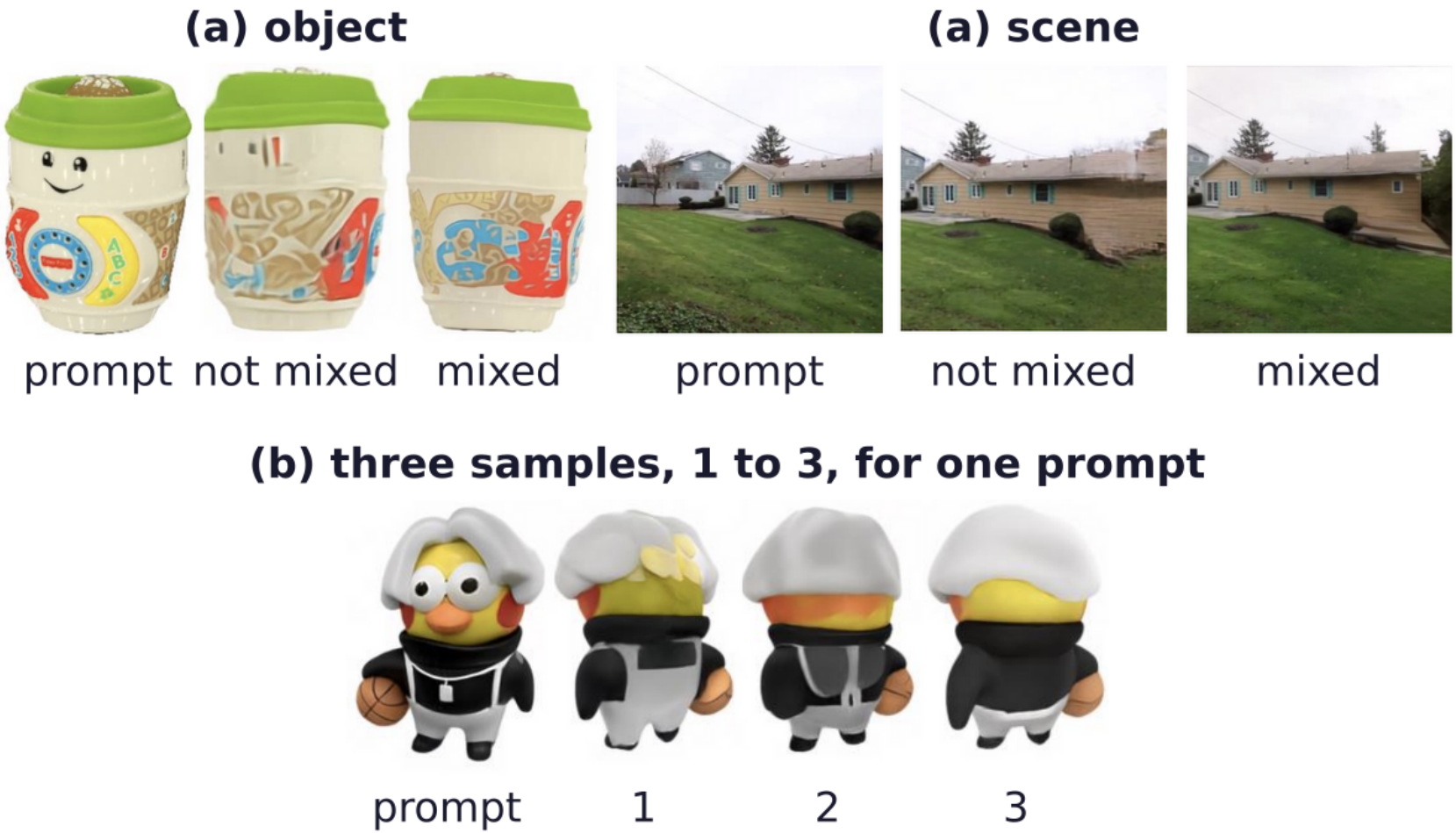


Figure 15: (Cai et al., 2025a, Figure 9.) (a) Novel views without and with mixed training; with it the cup's pattern and the house are sharper. (b) Three samples for one prompt: the unseen back differs between samples.

The project page, the code and the models are public

The project page shows the generated objects and scenes as videos and in interactive viewers:
`https://caiyuanhao1998.github.io/project/DiffusionGS/`

The code and the trained models, re-implemented after the paper, are at `https://github.com/caiyuanhao1998/Open-DiffusionGS`. For object-centric image-to-3D generation one script runs the model, `python run.py`; it downloads the model checkpoints and the configuration files itself.

The method needs one image and the relative camera poses of the views to generate, and about 6 seconds on one GPU. The two stages of earlier methods, view generation and reconstruction, are replaced by one diffusion model whose output is already 3D.

A video model generates 3D/4D with no 3D representation

DiffusionGS builds 3D into the generator. A different answer asks whether a 3D representation is needed at all. A video is a sequence of views of a scene; a video model that can be told where the camera moves generates the views of a 3D scene directly, and if the scene also moves, the views of a 4D scene (3D plus time).

OmniVCus (Cai et al., 2025c) customizes videos to a subject, a given object or character whose identity the video must keep, and is used in this way for 3D and 4D generation without any 3D representation: its videos keep the subject consistent while the camera circles it. Consistent views of a scene are one side of physical consistency; plausible motion over time is the other.

A video model generates 3D/4D with no 3D representation



Figure 16: One frame of the OmniVCus demonstration (Cai et al., 2025c): six generated subjects, each seen by a camera that circles it during the clip. The labels shorten the prompts, such as "a baby dragon hatching out of a stone egg".

Text-to-video generation turns a prompt into a video

Text-to-video (T2V) generation produces a video from a prompt, a sentence that describes it. Current models are diffusion models over the frames of a video: they denoise all the frames together, conditioned on the text, and the result looks real frame by frame.

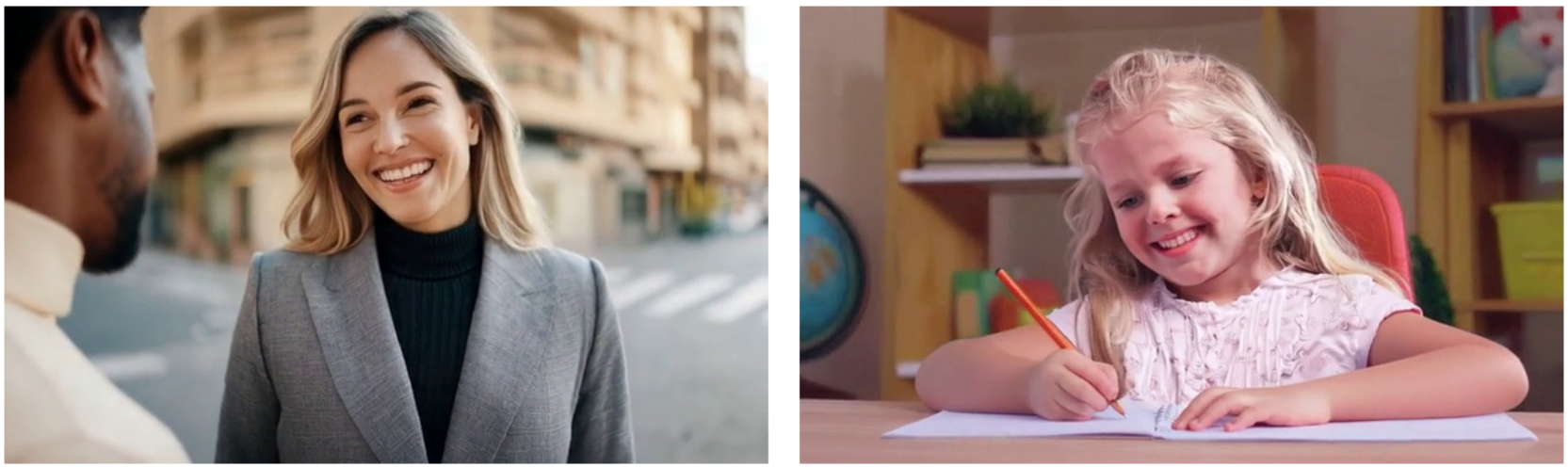


Figure 17: One frame of each of two generated videos. Prompts: (a) "A woman is talking to a man on a street." (b) "A child is happy doing her homework."

Even the best text-to-video models distort human bodies

Looking real frame by frame is a different thing from obeying physics over time. Motion that involves the whole body, contact and gravity is where the models fail. For the prompt “A gymnast drops from the parallel bars and lands safely on the mat below”, OpenAI’s Sora2 and Google DeepMind’s Veo3.1, among the best text-to-video models, generate distorted human bodies: at moments the gymnast’s body lies flat along the bar, blurred into it, and the drop and the landing do not follow a plausible trajectory. The physics is bad, even for the recent best models.

Even the best text-to-video models distort human bodies

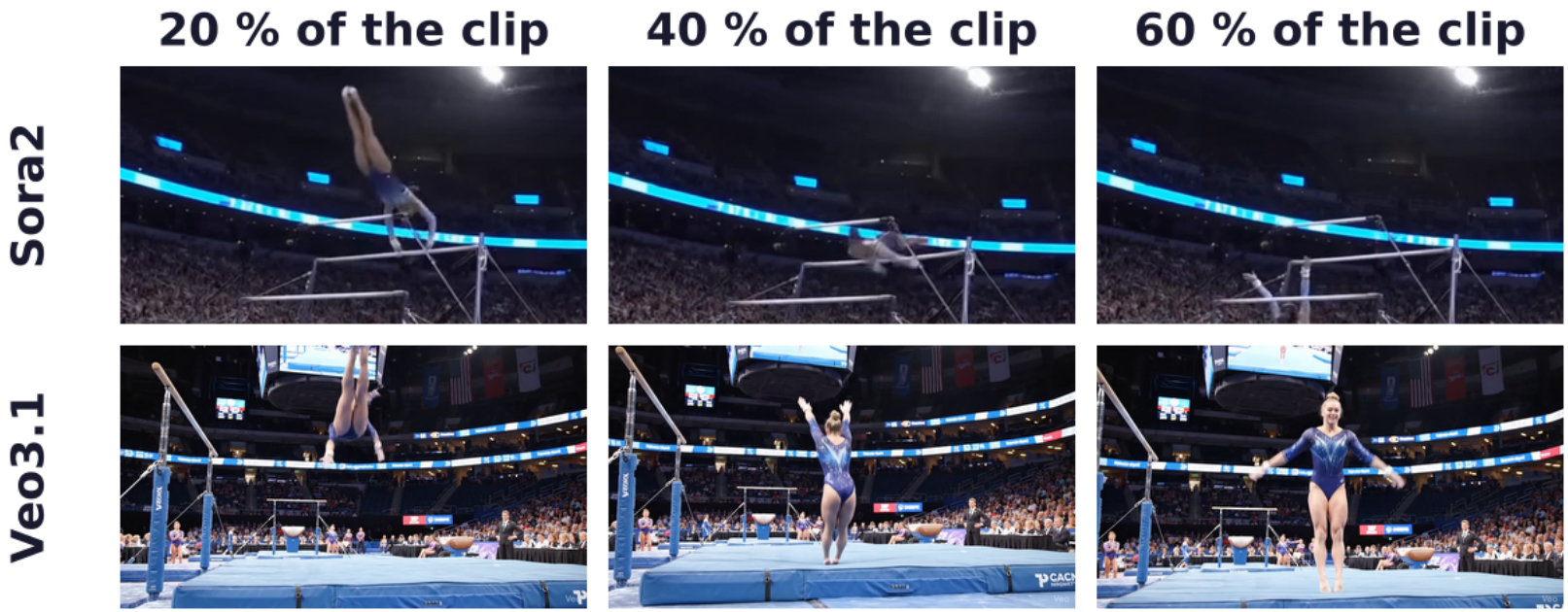


Figure 18: Frames of the two videos for the gymnast prompt, at 20 %, 40 % and 60 % of each clip. Sora2 (enlarged): at 40 % the body lies flat along the bar, blurred into it. Veo3.1: the gymnast is upside down above the bar, then already standing on the mat.

The goal: more physics, approaching a real-world simulator

The goal is to optimize a video model so that it has more physics, and so to approach a real-world simulator: a model that generates what would happen. The open model Wan2.1 (Team Wan et al., 2025), given the gymnast prompt, generates a physically inconsistent video. After fine-tuning with PhyGDPO, physics-aware groupwise direct preference optimization (Cai et al., 2025b), the same model generates a physically consistent one. Physically consistent videos can be used for human action analysis, where the motion itself is what is studied.

The goal: more physics, approaching a real-world simulator

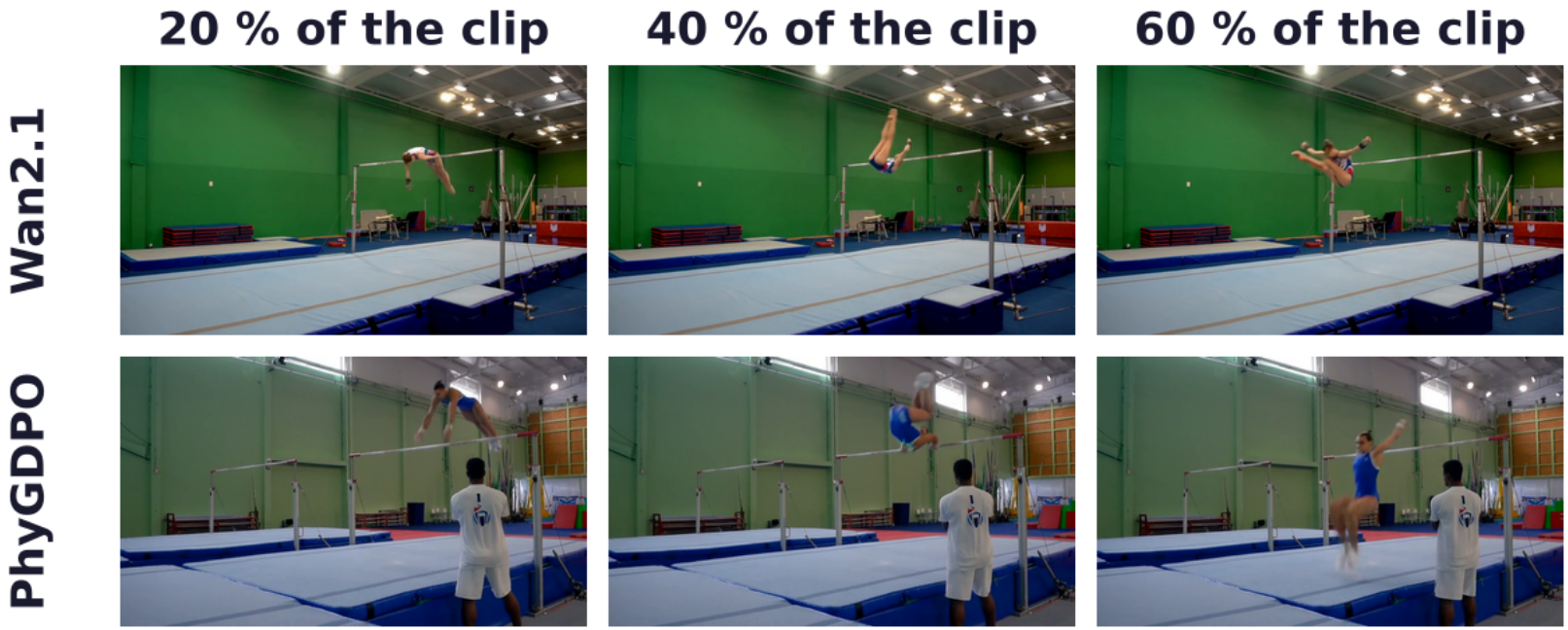


Figure 19: Frames of the two videos for the gymnast prompt, at 20 %, 40 % and 60 % of each clip. Wan2.1, physically inconsistent: the gymnast floats above the bar in poses with no support. Wan2.1 fine-tuned with PhyGDPO, physically consistent: the swing over the bar, then the drop to the mat.

Fine-tuning is costly, and plain DPO may face three problems

Text-to-video foundation models learn implicit physics from massive collections of high-quality text–video pairs, which are extremely costly to collect and annotate; models obtained by such supervised fine-tuning (SFT, training on given examples) still show limited physics reasoning, as Sora2 and Veo3.1 do.

Direct preference optimization (DPO; Rafailov et al., 2023) learns from comparisons instead: for a prompt, one output is preferred and one is rejected, and the model is trained to raise the probability of the preferred output relative to the rejected one, measured against a frozen copy of the model, the reference. Used directly, DPO may encounter three problems:

- too few training videos rich in physical actions, interactions and phenomena;
- its Bradley–Terry model, a model of the probability that one output is preferred to another, compares only pairs;
- a full copy of the model as the fixed reference takes much GPU memory and slows training.

PhyGDPO answers each problem in turn.

PhyAugPipe builds physics-rich training data

The first problem is data. PhyAugPipe, the physics-augmented data pipeline of PhyGDPO, builds training data rich in physics from a large pool of text–video pairs, about 1M.

A vision-language model (VLM), here Qwen, reads each video and its prompt and reasons step by step (chain of thought, CoT): it parses the elements of the scene, checks them in the video, reasons about the physics involved, scores the physics richness of the video and extends the prompt with the physical details. Videos with high physics richness are kept (about 135K pairs). A sentence transformer, a network that maps sentences to vectors, groups the prompts into categories of challenging actions, such as playing golf, somersaulting, a billiard shot or folding paper. Finally a physics-aware VLM, VideoCon-Physics (Bansal et al., 2025a), rates how well the text-to-video model performs on each action category, and the categories where it does poorly (the hard actions) are sampled more often. About 17K real videos are sampled for training.

PhyAugPipe builds physics-rich training data

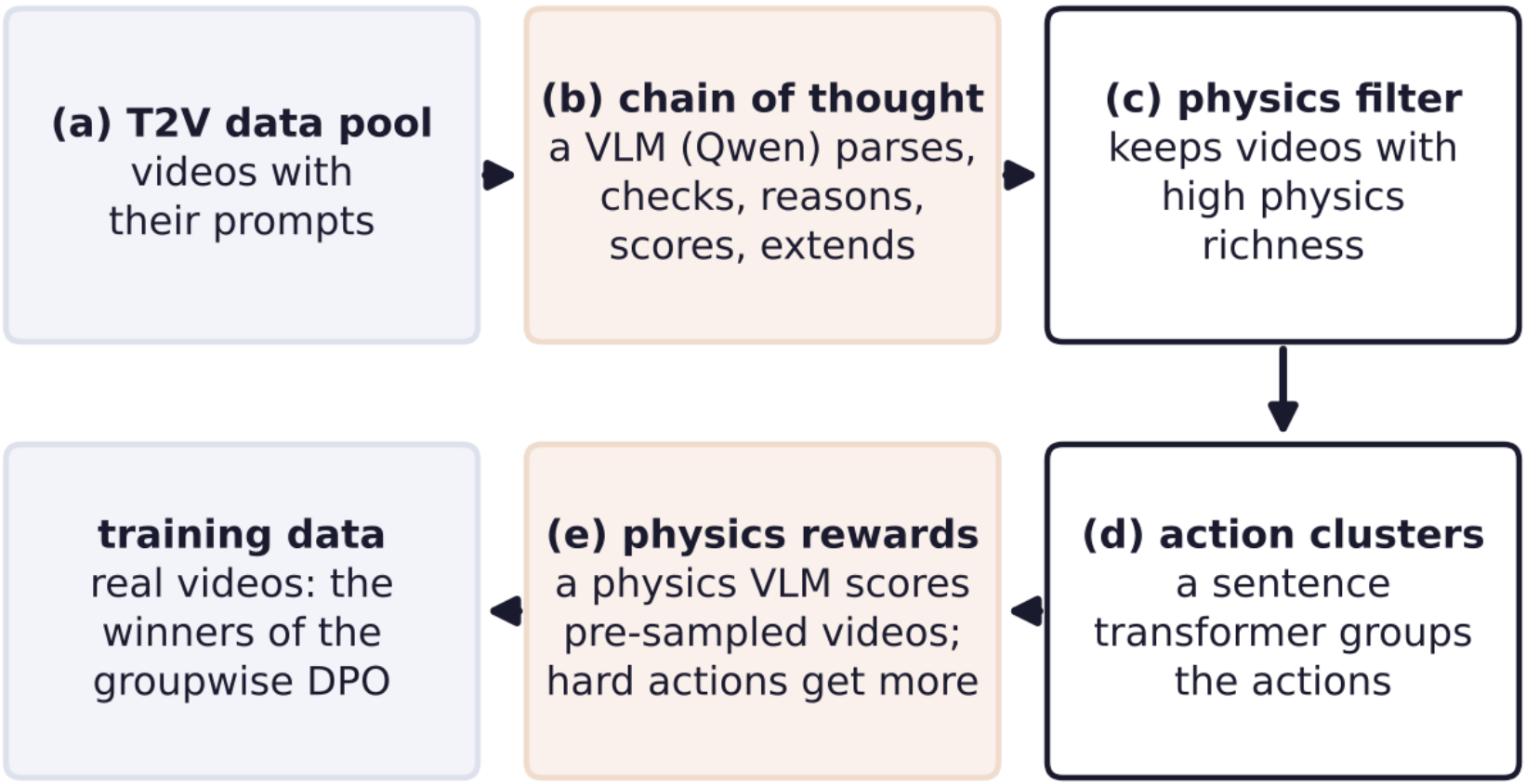


Figure 20: PhyAugPipe, drawn after Cai et al. (2025b), Figure 2: (a) the pool of text–video pairs; (b) the chain of thought of the VLM; (c) the filter on physics richness; (d) the clustering into action categories; (e) the sampling with physics rewards, which gives the real training videos.

A real video wins against a group of generated videos

The second problem is the pairwise comparison. PhyGDPO compares one winning video with a whole group of losing videos. For a prompt c the winning sample x_0^w is the real training video, and the losing samples $x_0^{l_1}, \ldots, x_0^{l_m}$ are videos the model generates for the same prompt from different random seeds: the group $\mathcal{G}^l(c)$ of the m losers; with the winner, $m+1$ videos are compared. The subscript 0 marks clean videos. (c and m are unrelated to the color $\mathbf{c}$ and the pixel row m of DiffusionGS.)

VideoCon-Physics scores every video on two counts in $[0, 1]$: SA, semantics adherence (does the video show what the prompt says), and PC, physics commonsense (does it obey physics). In the figure the real video scores 1 on both. The scores weight the losing samples in the loss, so the training pushes hardest against the videos that break physics most: physics-guided rewarding.

Finally, the text-to-video model serves as both the trained and the reference model, by switching modules of low-rank adaptation (LoRA) on and off: the LoRA-switch reference (LoRA-SR), which answers the third problem.

A real video wins against a group of generated videos



Figure 21: The groupwise comparison for a prompt about a woman kneeling on a beach who plays with a dog and a frisbee, with the four frames of Cai et al. (2025b), Figure 3(c). Blue: three videos generated from different seeds, the losing samples; red: the real video, the winning sample. Under each: its semantics-adherence (SA) and physics-commonsense (PC) scores. All four enter the groupwise Plackett–Luce model.

Plackett–Luce generalizes the Bradley–Terry model of DPO

In the Bradley–Terry model (Bradley and Terry, 1952), a video x for prompt c has a reward $r(c, x)$ (unrelated to the RPPC vector $\mathbf{r}$), and the probability that x_0^w is preferred to x_0^l is

$$p_{\mathrm{BT}}(x_0^w \succ x_0^l \mid c) = \frac{e^{r(c,x_0^w)}}{e^{r(c,x_0^w)} + e^{r(c,x_0^l)}} = \sigma\big(r(c, x_0^w) - r(c, x_0^l)\big),$$

with $\sigma(z) = 1/(1 + e^{-z})$ the logistic sigmoid, a third use of the letter σ.

The Plackett–Luce model (Plackett, 1975; Luce, 1959) ranks a whole set. The probability that the winner comes first among the $m + 1$ videos of a group is

$$p_{\mathrm{PL}}(x_0^w \mid \mathcal{G}^l(c), c) = \frac{\exp(r(c, x_0^w))}{\exp(r(c, x_0^w)) + \sum_{j=1}^{m} \exp(r(c, x_0^{l_j}))},$$

a softmax over the group. With $m = 1$ it is the Bradley–Terry model. Every losing video enters the denominator, so one comparison uses the whole group.

Plackett–Luce generalizes the Bradley–Terry model of DPO

In preference learning the model p_θ is trained to earn a high reward while staying close to a reference model p_ψ: it maximizes $\mathbb{E}[r(c,x)] - \beta\,\mathrm{KL}(p_\theta \| p_\psi)$, with KL the Kullback–Leibler divergence and $\beta > 0$ setting how close (this β is unrelated to the noise variances β_t). The maximizer is $p^*_\theta(x \mid c) = p_\psi(x \mid c)\exp(r(c,x)/\beta)/Z(c)$, $Z(c)$ the normalizing constant, so the reward can be written through the models, and DPO needs no separate reward model:

$$r(c, x_0) = \beta \log \frac{p^*_\theta(x_0 \mid c)}{p_\psi(x_0 \mid c)} + \beta \log Z(c).$$

Plackett–Luce generalizes the Bradley–Terry model of DPO

Put this reward into the Plackett–Luce model. $Z(c)$ is the same for every video of the prompt and cancels in p_{PL}. With the trained p_θ in place of p_θ^* and $f_\theta(x_0, c) = \log p_\theta(x_0 \mid c)/p_\psi(x_0 \mid c)$, the negative log-likelihood of the Plackett–Luce model is the GDPO (groupwise DPO) loss:

$$\mathcal{L}_{\mathrm{GDPO}}(\theta) = \mathbb{E}_{c,\mathcal{G}'(c)}\left[\log\left(1 + \sum_{j=1}^{m} \exp\left(\beta\left(f_\theta(x_0^{l_j}, c) - f_\theta(x_0^{w}, c)\right)\right)\right)\right].$$

It is small when the model, relative to the reference, makes the real video more likely than every generated one; with $m = 1$ it is the DPO loss.

An upper bound turns the group loss into weighted pairs

For a diffusion model the likelihood of a video is taken over its whole denoising chain $x_{t_T}, \ldots, x_{t_0} = x_0$, with timesteps $t_0 < \cdots < t_T$; for a real video the intermediate x_{t_k} come from the forward process. The likelihood of the chain is a product over its T steps, so the log-ratio is a sum (k now counts steps, no longer Gaussians):

$$f_\theta(x_0, c) = \sum_{k=1}^{T} \Delta_k, \qquad \Delta_k = \log \frac{p_\theta(x_{t_{k-1}} \mid x_{t_k}, c)}{p_\psi(x_{t_{k-1}} \mid x_{t_k}, c)}.$$

The sum is T times the mean over a random step k, and $\log(1 + \sum_j e^{z_j})$ is convex, so by Jensen's inequality

$$\mathcal{L}_{\mathrm{GDPO}}(\theta) \le \mathbb{E}_{c, \mathcal{G}^I(c), k}\left[\log\left(1 + \sum_{j=1}^{m} \exp\left(\beta T [\Delta_k^{l_j} - \Delta_k^{w}]\right)\right)\right].$$

Training then needs one random step per video, as in diffusion training.

An upper bound turns the group loss into weighted pairs

A second inequality splits the group into pairs: for $0 < \alpha_j \leq 1$ and $\gamma_j \geq 1/\alpha_j$,

$$1 + \sum_{j=1}^{m} e^{z_j} \leq \prod_{j=1}^{m} \left(1 + e^{\alpha_j z_j}\right)^{\gamma_j}.$$

Each factor is at least $1 + e^{z_j}$, since $(1 + e^{\alpha z})^{\gamma} \geq (1 + e^{\alpha z})^{1/\alpha} \geq 1 + e^{z}$; and the product of the factors $1 + e^{z_j}$, multiplied out, contains 1 and every e^{z_j}. The middle step holds because $(1 + u)^p \geq 1 + u^p$ for $u > 0$ and $p = 1/\alpha \geq 1$, with $u = e^{\alpha z}$.

Applied with $z_j = \beta T[\Delta_k^{l_j} - \Delta_k^{w}]$, after taking logarithms, it gives the bound used for training,

$$\mathcal{L}_{\mathrm{GDPO}}(\theta) \leq \mathbb{E}_{c, \mathcal{G}^{I(c),k}} \left[\sum_{j=1}^{m} \gamma_j \log \left(1 + \exp(-\alpha_j \beta T[\Delta_k^{w} - \Delta_k^{l_j}])\right) \right].$$

An upper bound turns the group loss into weighted pairs

Since $\log(1+e^{-z}) = -\log\sigma(z)$ and a sum over j is m times the mean over a random j, the bound reads

$$\mathcal{L}_{\mathrm{GDPO}}(\theta) \le \mathbb{E}_{c,\mathcal{G}^{I(c)},k,j}\big[-m\gamma_j \log\sigma\big(\alpha_j\beta\,T(\Delta_k^w - \Delta_k^{l_j})\big)\big].$$

Each term is a DPO loss for one pair, the real video against loser j, with its own weight γ_j and sharpness α_j. The weights come from the physics scores of the losing videos:

$$v_j = 1 - \frac{s_j^{sa} + s_j^{pc}}{2}, \qquad \gamma_j = \frac{1+\lambda\,\sigma(\kappa_\gamma(v_j - b_\gamma))}{\alpha_{\min}},$$

$$\alpha_j = \alpha_{\min} + (1-\alpha_{\min})\,\sigma(\kappa_\alpha(v_j - b_\alpha)).$$

v_j is the physics difficulty of loser j, high when its SA and PC scores s_j^{sa}, s_j^{pc} are low; worse videos get larger γ_j and α_j. The hyperparameters are $\alpha_{\min} = 0.5$, $\lambda = 0.6$, $\kappa_\gamma = 2.0$, $b_\gamma = 0.4$, $\kappa_\alpha = 5.0$, $b_\alpha = 0.5$. Since $\alpha_j \ge \alpha_{\min}$, $\gamma_j \ge 1/\alpha_{\min} \ge 1/\alpha_j$, as the bound requires. (α_j and λ here are unrelated to the opacity and the VGG weight of DiffusionGS.)

LoRA-Switch Reference shares one backbone for both models

The third problem is memory. Every term of the loss needs both the trained model θ and the reference ψ, and vanilla DPO keeps two full copies of a large video model.

LoRA (low-rank adaptation; Hu et al., 2022) fine-tunes a large network by adding to a frozen weight matrix $\mathbf{W}$ a product of two thin matrices, $\mathbf{W} + \frac{a}{r}\mathbf{BA}$, with rank r (a fourth meaning of r) much smaller than the size of the weight matrix $\mathbf{W}$ and a scale a; only $\mathbf{A}$ and $\mathbf{B}$ are trained. In the LoRA-switch reference (LoRA-SR) the backbone is frozen as the reference model ψ, and the trainable LoRA modules attached to it make the trained model θ. Switching the lightweight LoRA parameters on gives the trained model, switching them off gives the reference. Both terms of the loss are evaluated without storing or loading an extra full model. This saves GPU memory, speeds up training, and keeps the model close to the reference, which stabilizes training.

LoRA-Switch Reference shares one backbone for both models

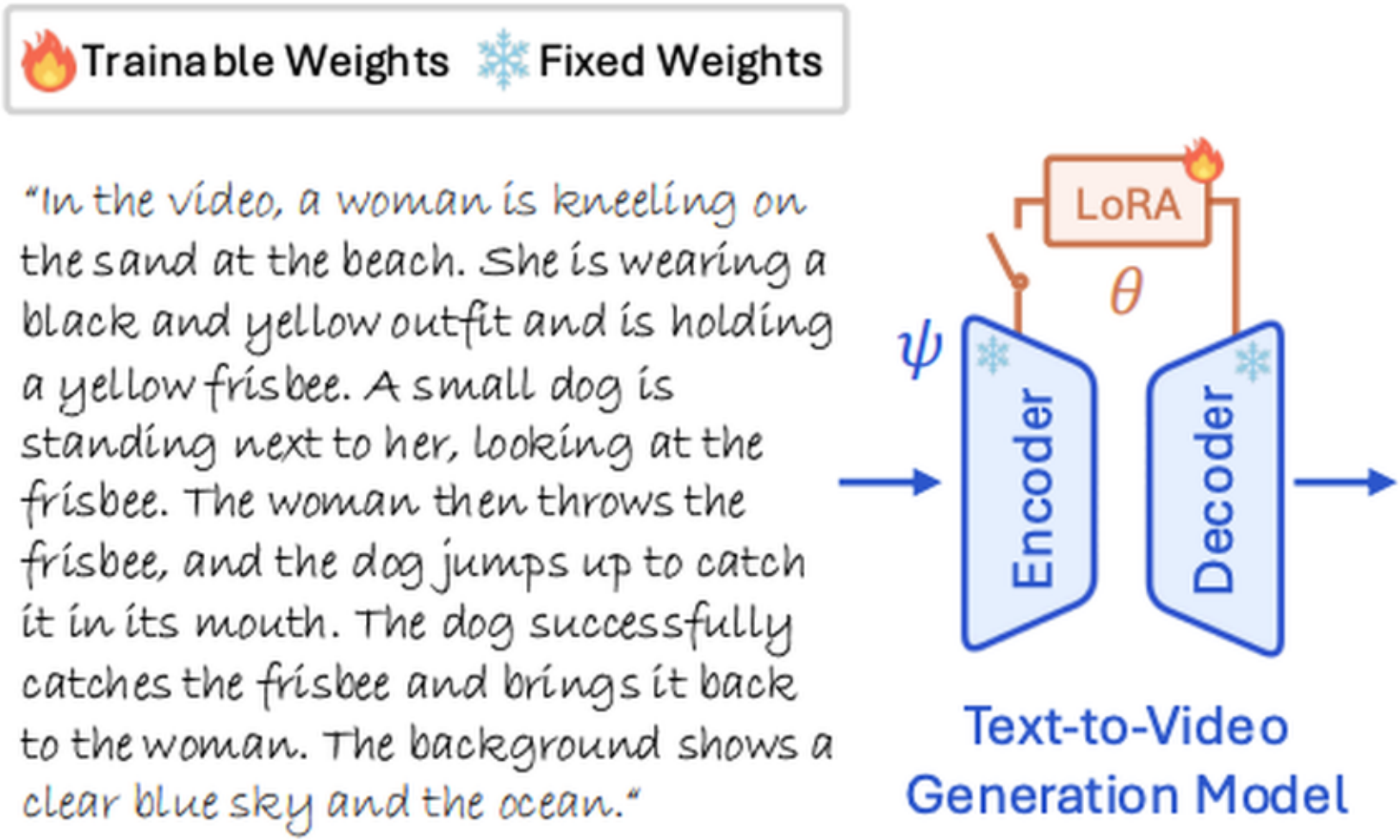


Figure 22: Cai et al. (2025b), Figure 3 (a), (b): a prompt, and the frozen backbone (the reference ψ) with a LoRA branch behind a switch (on: θ).

PhyGDPO raises the physics scores of Wan2.1

PhyGDPO fine-tunes Wan2.1-T2V-14B, the 14-billion-parameter text-to-video model of Wan2.1. On VideoPhy2 (Bansal et al., 2025b), a benchmark of physical actions whose score is the fraction of videos that its own VLM judge rates at least 4 of 5 for both semantics adherence and physics commonsense (a different judge and scale from the training rewards), the score rises from 0.1288 to 0.1627, above Veo3.1 (0.1525) and Sora2 (0.1508); on its hard actions it rises from 0.0111 to 0.0500. On PhyGenBench (Meng et al., 2025), a benchmark of physical phenomena, it rises from 0.40 to 0.55, against 0.50 for PhyT2V and 0.54 for VideoDPO. In a user study the PhyGDPO videos were preferred to those of Veo3.1 in 64.4 % and to those of Sora2 in 67.3 % of the comparisons (Cai et al., 2025b, arXiv version 4).

Four sports prompts compare the videos of six models: PhyT2V (Xue et al., 2025), which refines the prompt with a large language model to ground it in physics; VideoDPO (Liu et al., 2025), which applies DPO to video models for visual quality and text alignment; Wan2.1, the base model; Sora2; Veo3.1; and Wan2.1 fine-tuned with PhyGDPO.

Soccer: PhyGDPO against five text-to-video models

Prompt: “A soccer player runs, plants their foot, and drop kicks a soccer ball high into the air, the ball arcing visibly.” The action asks for a run-up, a planted foot, contact with the ball and its arc.



Figure 23: One frame of each video, at 15 % of the clip. PhyT2V shows two balls at once; in VideoDPO and Sora2 the player runs with the ball at the feet; in Veo3.1 and PhyGDPO the ball is in the air, kicked up by the player.

Basketball: PhyGDPO against five text-to-video models

Prompt: “A player dunks the basketball, the basketball soaring upward before slamming through the net.” The action asks for a jump, the ball carried up to the rim and pushed down through the net.

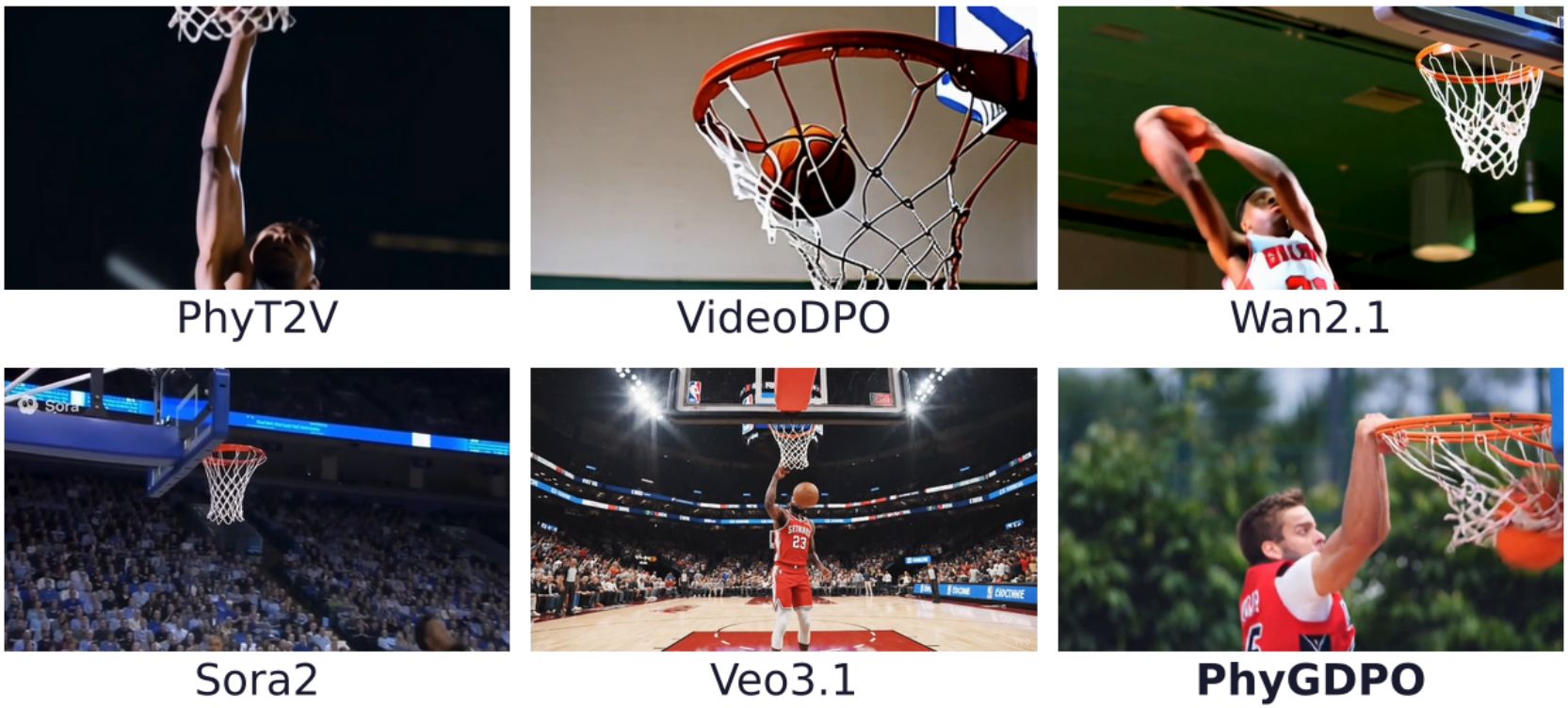


Figure 24: One frame of each video, at 45 % of the clip. In PhyGDPO the player hangs at the rim as the ball goes through the net; in Wan2.1 the player rises with the ball; Sora2 shows an empty basket.

Squash: PhyGDPO against five text-to-video models

Prompt: “A person plays squash on an indoor court.” Squash needs the player, the racket and the ball to stay coherent while the player runs and swings, inside the walls of the court.

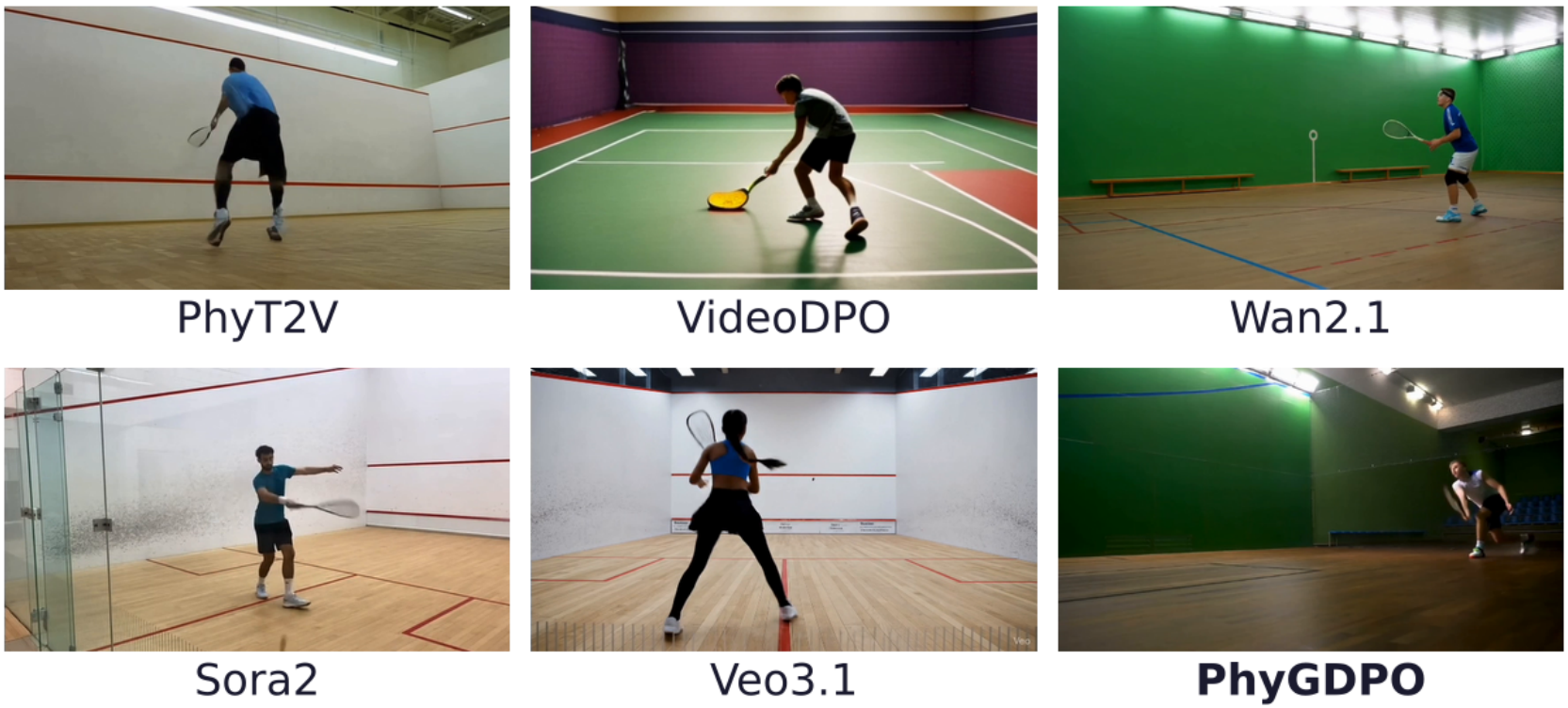


Figure 25: One frame of each video, at the middle of the clip. In VideoDPO the racket has turned into a flat yellow paddle on the floor; Sora2 and PhyGDPO show the player in the middle of a stroke.

Polo: the mallet striking the ball is the hard interaction

Prompt: “A player rides their horse, preparing to strike the ball, their mallet poised.” The comparison video notes that the other methods never show the mallet reaching the ball; in PhyGDPO it does.



Figure 26: One frame of each video, at 34 % of the clip, the moment of the strike in PhyGDPO: the head of its mallet meets the ball in the air. VideoDPO shows two balls; in the other videos the mallet and a ball do not meet.

Each ablation step adds one component of PhyGDPO

The ablation uses the prompt "A tennis ball is gently placed on the surface of a bucket filled with water." A tennis ball floats: it should rest on the water, partly submerged, and make ripples. Four models are compared, each adding one component to the one before: Wan2.1 alone; with LoRA-SR training; with the groupwise Plackett–Luce model; and with the physics rewards. Wan2.1 alone lets the ball vanish under the water and reappear in the air; on this prompt each component makes the motion more stable, and with all three the ball rests on the surface, partly submerged, as a floating ball should.

Each ablation step adds one component of PhyGDPO

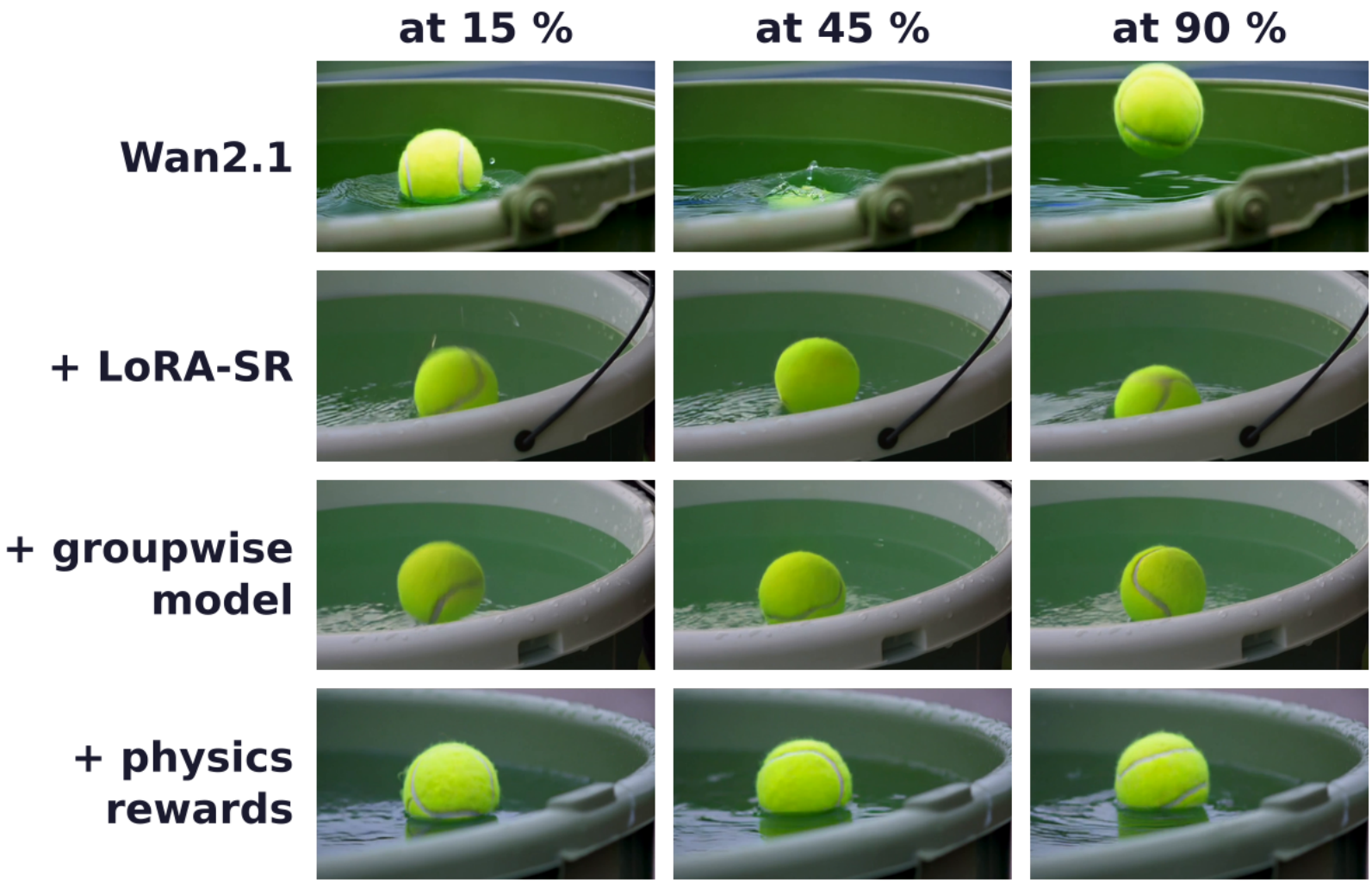


Figure 27: The four videos (rows) at 15 %, 45 % and 90 % of the clip. Wan2.1: the ball floats, vanishes in a splash, then hangs in the air. + LoRA-SR: it bobs up and down. + groupwise model: it stays at the surface. + physics rewards: it rests on the water, partly submerged.

Summary

- Image-to-3D generation: one view in, a 3D representation out, rendered from new views.
- Two-stage methods generate views first and reconstruct 3D afterwards; the views disagree.
- DiffusionGS: a one-stage diffusion model whose denoiser predicts 3D Gaussians at every step.
- Supervised by 2D renderings it needs no 3D ground truth; mixed training and RPPC help.
- It generates objects and scenes from one view in seconds, with better scores than earlier methods.
- Text-to-video models still break physics; plain DPO compares only pairs and copies the whole model.
- PhyGDPO: physics-rich data, a groupwise Plackett–Luce loss with VLM rewards, and LoRA-SR.
- Sports prompts compare PhyGDPO with Sora2, Veo3.1, Wan2.1, PhyT2V and VideoDPO.

References

[1] Y. Cai, H. Zhang, K. Zhang et al. Baking Gaussian splatting into diffusion denoiser for fast and scalable single-stage image-to-3D generation and reconstruction. *ICCV*, 25062–25072, 2025 (2025a).

[2] Y. Cai, K. Li, M. Jia et al. PhyGDPO: physics-aware groupwise direct preference optimization for physically consistent text-to-video generation. arXiv:2512.24551, 2025 (2025b).

[3] Y. Cai, H. Zhang, X. Chen et al. OmniVCus: feedforward subject-driven video customization with multimodal control conditions. *NeurIPS*, 115404–115423, 2025 (2025c).

[4] B. Mildenhall, P. P. Srinivasan, M. Tancik et al. NeRF: representing scenes as neural radiance fields for view synthesis. *ECCV*, 405–421, 2020.

[5] B. Kerbl, G. Kopanas, T. Leimkühler and G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):1–14, 2023.

[6] R. Gao, A. Hołyński, P. Henzler et al. CAT3D: create anything in 3D with multi-view diffusion models. *NeurIPS*, 75468–75494, 2024.

[7] J. Tang, Z. Chen, X. Chen et al. LGM: large multi-view Gaussian model for high-resolution 3D content creation. *ECCV*, 1–18, 2024 (2024a).

References

[8] M. Liu, R. Shi, L. Chen et al. One-2-3-45++: fast single image to 3D objects with consistent multi-view generation and 3D diffusion. *CVPR*, 10072–10083, 2024.

[9] H. Wang, Y. Liu, Z. Liu, W. Wang, Z. Dong and B. Yang. VistaDream: sampling multiview consistent images for single-view scene reconstruction. *ICCV*, 26772–26782, 2025.

[10] S. Szymanowicz, C. Rupprecht and A. Vedaldi. Splatter Image: ultra-fast single-view 3D reconstruction. *CVPR*, 10208–10217, 2024.

[11] J. Ho, A. Jain and P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 6840–6851, 2020.

[12] J. Song, C. Meng and S. Ermon. Denoising diffusion implicit models. *ICLR*, 2021.

[13] M. Deitke, D. Schwenk, J. Salvador et al. Objaverse: a universe of annotated 3D objects. *CVPR*, 13142–13153, 2023.

[14] X. Yu, M. Xu, Y. Zhang et al. MVImgNet: a large-scale dataset of multi-view images. *CVPR*, 9150–9161, 2023 (2023a).

References

[15] T. Zhou, R. Tucker, J. Flynn, G. Fyffe and N. Snavely. Stereo magnification: learning view synthesis using multiplane images. *ACM Transactions on Graphics*, 37(4):65:1–65:12, 2018.

[16] L. Ling, Y. Sheng, Z. Tu et al. DL3DV-10K: a large-scale scene dataset for deep learning-based 3D vision. *CVPR*, 22160–22169, 2024.

[17] V. Sitzmann, S. Rezchikov, W. T. Freeman, J. B. Tenenbaum and F. Durand. Light field networks: neural scene representations with single-evaluation rendering. *NeurIPS*, 19313–19325, 2021.

[18] G. Zhang, R. Zha, H. He, Y. Liang, A. Yuille, H. Li and Y. Cai. X-LRM: X-ray large reconstruction model for extremely sparse-view computed tomography recovery in one second. arXiv:2503.06382, 2025.

[19] Y. Xu, H. Tan, F. Luan et al. DMV3D: denoising multi-view diffusion using 3D large reconstruction model. *ICLR*, 2024.

[20] Z. Wang, Y. Wang, Y. Chen et al. CRM: single image to 3D textured mesh with convolutional reconstruction model. *ECCV*, 57–74, 2024.

References

[21] J. Tang, J. Ren, H. Zhou, Z. Liu and G. Zeng. DreamGaussian: generative Gaussian splatting for efficient 3D content creation. *ICLR*, 2024 (2024b).

[22] J. Collins, S. Goel, K. Deng et al. ABO: dataset and benchmarks for real-world 3D object understanding. *CVPR*, 21126–21136, 2022.

[23] L. Downs, A. Francis, N. Koenig et al. Google Scanned Objects: a high-quality dataset of 3D scanned household items. *ICRA*, 2553–2560, 2022.

[24] K. Zhang, S. Bi, H. Tan et al. GS-LRM: large reconstruction model for 3D Gaussian splatting. *ECCV*, 1–19, 2024.

[25] S. Szymanowicz, E. Insafutdinov, C. Zheng et al. Flash3D: feed-forward generalisable 3D scene reconstruction from a single image. arXiv:2406.04343, 2024; *3DV*, 2025.

[26] A. Yu, V. Ye, M. Tancik and A. Kanazawa. pixelNeRF: neural radiance fields from one or few images. *CVPR*, 4578–4587, 2021.

[27] J. J. Yu, F. Forghani, K. G. Derpanis and M. A. Brubaker. Long-term photometric consistent novel view synthesis with diffusion models. *ICCV*, 7094–7104, 2023 (2023b).

References

[28] Team Wan, A. Wang, B. Ai et al. Wan: open and advanced large-scale video generative models. arXiv:2503.20314, 2025.

[29] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning and C. Finn. Direct preference optimization: your language model is secretly a reward model. *NeurIPS*, 53728–53741, 2023.

[30] H. Bansal, Z. Lin, T. Xie et al. VideoPhy: evaluating physical commonsense for video generation. *ICLR*, 2025 (2025a).

[31] R. A. Bradley and M. E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

[32] R. L. Plackett. The analysis of permutations. *Applied Statistics*, 24(2):193–202, 1975.

[33] R. D. Luce. *Individual Choice Behavior: A Theoretical Analysis*. Wiley, New York, 1959.

[34] E. J. Hu, Y. Shen, P. Wallis et al. LoRA: low-rank adaptation of large language models. *ICLR*, 2022.

References

[35] H. Bansal, C. Peng, Y. Bitton et al. VideoPhy-2: a challenging action-centric physical commonsense evaluation in video generation. arXiv:2503.06800, 2025 (2025b); *ICLR*, 2026.

[36] F. Meng, J. Liao, X. Tan et al. Towards world simulator: crafting physical commonsense-based benchmark for video generation. *ICML*, 2025.

[37] Q. Xue, X. Yin, B. Yang and W. Gao. PhyT2V: LLM-guided iterative self-refinement for physics-grounded text-to-video generation. *CVPR*, 18826–18836, 2025.

[38] R. Liu, H. Wu, Z. Zheng et al. VideoDPO: omni-preference alignment for video diffusion generation. *CVPR*, 8009–8019, 2025.